

**MACHINE LEARNING BASED ALGORITHM
SELECTION FOR SORTING NUMERIC ARRAYS**

AHMED SALAH ABDALHI MOHAMMED

**A THESIS FOR
THE DEGREE OF MASTER OF SCIENCE (MSc)
IN
COMPUTER ENGINEERING**



**CYPRUS INTERNATIONAL UNIVERSITY
INSTITUTE OF GRADUATE STUDIES AND RESEARCH**

NICOSIA - 2026

**MACHINE LEARNING BASED ALGORITHM
SELECTION FOR SORTING NUMERIC ARRAYS**

**A THESIS SUBMITTED TO
THE INSTITUTE OF GRADUATE STUDIES AND RESEARCH
OF
CYPRUS INTERNATIONAL UNIVERSITY**

**BY
AHMED SALAH ABDALHI MOHAMMED**

**IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF MASTER OF SCIENCE (MSc)
IN
COMPUTER ENGINEERING**

SUPERVISOR : ASSOC. PROF. DR. EMRE ÖZBİLGE

NICOSIA - 2026

TABLE OF CONTENTS

CHAPTER ONE	6
INTRODUCTION	6
1.1 Problem Statement	6
1.2 Research Objectives	8
1.3 Scope of the Research	8
1.4 Research Significance	9
1.5 Research Organisation.....	9
CHAPTER TWO	10
LITERATURE REVIEW.....	10
2.1 Purpose and Scope of the Literature Review	10
2.2 Organization of the Chapter	10
2.3 Sorting Algorithms and Input-Dependent Performance	10
2.3.1 Comparison-Based and Non-Comparison-Based Sorting	11
2.3.2 Lower Bound of Comparison-Based Sorting	11
2.3.4 Quicksort	12
2.3.5 Introsort	12
2.3.6 Heapsort.....	13
2.3.7 Mergesort.....	13
2.3.8 Timsort and Run-Adaptive Sorting	13
2.3.9 Shell Sort	14
2.4 Input Structure and Sorting Performance	15
2.4.1 Array Size and Runtime Scaling.....	15
2.4.2 Sortedness and Disorder	15
2.4.3 Runs and Natural Order.....	15
2.4.4 Duplicates and Repeated Values.....	16
2.4.5 Entropy and Value Diversity.....	16
2.4.6 Distribution, Spread, Skewness, and Outliers	16
2.5 The Algorithm Selection Problem	17
2.5.1 Rice’s Algorithm Selection Framework	17
2.5.2 Mapping Rice’s Framework to Sorting Algorithm Selection	18

2.5.3 Per-Instance Algorithm Selection	19
2.5.4 Algorithm Portfolios and Selection Mappings	19
2.5 Machine Learning-Based Solver and Algorithm Selection	20
2.5.1 SATzilla and Feature-Based Solver Selection	20
2.5.2 Empirical Performance Models	21
2.5.3 Classification-Based Selection	21
2.5.4 Runtime Regression and Ranking-Based Selection	21
2.5.5 Relevance to Sorting Algorithm Selection	22
2.6 Feature-Based Instance Characterisation	22
2.6.1 Motivation for Structural Array Features	22
2.6.2 Size Features.....	24
2.6.3 Sortedness and Disorder Features.....	24
2.6.4 Run-Based Features.....	25
2.6.5 Duplicate and Frequency Features	26
2.6.6 Entropy Feature	27
2.6.7 Dispersion and Robust Spread Features	28
2.6.8 Shape and Outlier Features.....	29
2.6.9 Summary of Feature Groups	31
2.7 Supervised Learning for Sorting Algorithm Selection	33
2.7.1 Formulating the Task as Supervised Learning.....	33
2.7.2 Runtime-Based Label Generation	34
2.7.3 XGBoost for Tabular Algorithm Selection	34
2.7.4 Feature Importance and Model Interpretability.....	35
2.7.5 Timing Noise and Benchmarking Reliability	36
2.8 Adaptive, Hybrid, and Learned Sorting.....	38
2.8.1 Adaptive Sorting.....	38
2.8.2 Hybrid Sorting and Introsort	38
2.8.3 Adaptive Hybrid Sort.....	38
2.9 Prediction-Based and Learning-Augmented Sorting	39
2.9.1 Learning-Augmented Algorithms.....	39
2.9.2 Sorting with Predictions	39
2.9.3 AlphaDev	40
2.9.4 Algorithm Discovery vs Algorithm Modification vs Algorithm Selection	41
2.10 Real-World Numeric Arrays and Domain-Specific Patterns.....	41
2.10.1 Synthetic Arrays for Controlled Evaluation	41

2.10.2 Real-World Numeric Arrays	42
2.10.3 Formula 1 Telemetry.....	43
2.10.4 Finance, Weather, Cryptocurrency, and Seismic Data.....	44
2.10.5 Relationship Between Domain Structure and Feature Values	46
2.11 Domain Generalisation and Routed Models.....	48
2.11.1 Motivation for Domain Generalisation.....	48
2.11.2 Within-Domain and Cross-Domain Evaluation.....	49
2.11.3 Leave-One-Domain-Out Testing	50
2.11.4 General, Domain-Specific, and Routed Specialised Models.....	51
2.11.5 Routed Specialised Models for F1 Telemetry.....	53
2.12 Evaluation Metrics for Algorithm Selection	55
2.12.1 Why Accuracy Alone Is Insufficient.....	55
2.12.2 Single Best Solver	55
2.12.3 Virtual Best Solver.....	56
2.12.4 Regret	56
2.12.5 Gap Closed	56
2.12.6 Summary of Evaluation Metrics.....	56
CHAPTER THREE	58
METHODOLOGY	58
3.1 Research Methodology Overview	58
3.2 System Architecture	58
3.3 Algorithm Portfolio	59
3.4 Feature Engineering	60
3.4.1 Numerical Stability and Normalisation.....	61
3.5 Data Collection and Labelling.....	61
3.5.1 Structural Transforms	62
3.5.2 Labelling Protocol	62
3.5.3 Quality Controls	63
3.6 XGBoost Classifier.....	63
3.6.1 Class Imbalance Handling.....	63
3.6.2 Training Protocol	64
3.6.3 Learnt Structure	64
3.7 Evaluation Protocol	66

3.7.1 Classification Metrics	66
3.7.2 Regret Metrics	66
3.7.3 Baseline Comparisons	68
3.7.4 Domain Holdout Evaluation.....	68
3.8 Reproducibility	68
3.9 Conclusion.....	69
4.1 Overview	70
4.2 Production Model Performance.....	70
4.3 Interpretation of Model Iterations	71
4.4 The Binary Nature of the Selection Problem	72
4.5 Generalisation Across Domains	72
4.6 Routed Models and Source Leakage	73
4.7 Implications of the Findings.....	74
4.9 Chapter Summary.....	76
REFERENCES	81

CHAPTER ONE

INTRODUCTION

Sorting can be defined as a foundational operation in computer science that involves a building block of database engines, analytics platforms, and machine-learning pipelines. Because sorting is executed continuously and at a large scale, measurable reductions in latency, lower energy consumption and infrastructural cost could be achieved by even small improvements in sorting (Mankowitz et al., 2023a).

However, there is no definitive sorting algorithm that performs best for inputs. Structural properties of the data such as array size, degree of presortedness, duplicate concentration, and distribution shape play an important role in runtime behaviour (Wiredu, Baagyere, et al., 2025). These properties interact with low-level effects such as cache locality, branching, and memory access patterns, so algorithms with the same asymptotic complexity often differ substantially in real execution time (Kassabgi et al., 2025).

This thesis addresses such variability through per-instance algorithm selection. Instead of applying a single fixed algorithm to every input, the proposed system predicts the most efficient algorithm for each array at runtime. It combines an offline machine-learning predictor for immediate decisions with an online adaptation layer that refines selection from observed execution outcomes.

The central argument of this chapter is that runtime selection is only valuable when prediction quality, decision overhead, and adaptability are treated as a single design problem. Chapter One therefore defines the problem, formalises the optimisation objective, states the research aims, and establishes the scope and significance of the study.

1.1 PROBLEM STATEMENT

Arrays drawn from many domains, each with distinct statistical structure are processed using modern computing workloads. Default algorithm choices are therefore unreliable across mixed workloads. A selector that adapts the choice of algorithm per

input has clear potential to improve performance only if the cost of making the decision remains smaller than the runtime it saves.

Three interacting constraints are created by this requirement. First, efficiency: feature extraction and prediction must be computationally lightweight. Second, latency: decisions must be produced fast enough to integrate into production pipelines. Third, resilience: for the selector to remain effective incoming data must differ from the data used in training the sector. Various constraints often cause conflict in practice because stronger predictive models tend to be more expensive to evaluate and are generalised poorly under distribution shift.

The research problem is to design a practical selector that helps in improving end-to-end runtime under real deployment conditions and not simply to identify the fastest algorithm in theory. The selector must be able to deliver factual gains, that are cost effective to justify its overhead, and adaptive enough to remain reliable as workloads progresses.

Observing it in a technical approach, let X be equal to the space of possible input arrays and $A = \{a_1, a_2, \dots, a_k\}$ candidate sorting algorithms collection. For an input $x \in X$, let $T(a, x)$ denote the runtime of algorithm a on x . The oracle selector chooses $a^* = \text{argmin}_a T(a, x)$; however, computing this oracle requires evaluating every algorithm and is therefore impractical at inference time.

A deployable selector S maps instance features $f(x)$ to a single algorithm in A , that is, $S: f(x) \rightarrow A$. The objective is to reduce expected regret with respect to the oracle while accounting for the selector's own overhead. The per-instance regret is defined in equation 1 as:

$$R(x) = \frac{\left(T(S(f(x)), x) - T(a^*, x)\right)}{T(a^*, x)} \quad (1)$$

When the total cost of feature extraction, prediction, and execution of the chosen algorithm is lower than that of an appropriate fixed baseline, a selector is therefore useful.

Two practical realities of runtime selection are highlighted by this formulation. First, the most crucial being time-based utility because classification accuracy can not be used as a sufficient quality metric. Second, when several algorithms are near-tied on many inputs, the effective decision boundary becomes clustered, which motivates online adaptation after deployment.

1.2 RESEARCH OBJECTIVES

The main aim of this thesis is to design, implement, and validate an adaptive runtime sorting algorithm selector for real-world data. The specific objectives are:

- Define and validate a compact set of structural array features that are informative for algorithm selection and computationally inexpensive to extract.
- Train and evaluate an offline XGBoost model for per-instance algorithm selection.
- Quantify the practical performance ceiling of static selection under a rigorous, leakage-resistant evaluation protocol.
- Implement an online contextual bandit layer that adapts selection decisions under distribution shift.
- Package the complete method as a reproducible Python library suitable for deployment and further research.

1.3 SCOPE OF THE RESEARCH

Comparison-based sorting selection within NumPy’s C-level implementations, specifically introsort, heapsort, and timsort is the focus of this thesis. Feature design, offline model training, online adaptation, and evaluation across multiple real-world domains are covered. Non-comparison sorts and custom low-level reimplementations in C or C++ are excluded so that observed performance differences reflect selection strategy and data structure rather than implementation optimisations.

Evaluation is performed under strict source-level separation between training, validation, and testing data to reduce leakage and discourage overfitting to dataset-specific artifacts. End-to-end runtime outcomes is emphasized in this study rather than

classification metrics alone, and explicitly examines behaviour under distribution shift in order to approximate production conditions.

1.4 RESEARCH SIGNIFICANCE

There are three main ways in which this research contributes to algorithm selection. First, a practical framework is provided for sorting selection that combines statistical prediction with online adaptation, closing the gap between static offline models and dynamic production environments in which workload characteristics evolve over time.

Second, a more rigorous evaluation perspective for sorting selection by prioritising source-level split integrity, cost-aware performance metrics, and explicit treatment of near-tied algorithmic outcomes is introduced this thesis. The credibility and reproducibility of reported runtime gains is improved.

Third, the work has clear practical relevance. Even small per-instance improvements compound at scale, reducing cloud computing cost, improving throughput, and lowering latency in data-intensive systems. This thesis translates these methodological advances into an accessible engineering tool by delivering the approach as a reproducible Python library.

1.5 RESEARCH ORGANISATION

This thesis is grouped into five chapters. Chapter One introduces the problem, formal definition, objectives, scope, and significance of the research. Chapter Two reviews the theoretical background and related work on sorting algorithms, algorithm selection, feature engineering, gradient-boosted trees, and contextual bandits. Chapter Three presents the methodology, including system architecture, data collection, feature design, modelling, and evaluation protocol. Chapter Four reports and analyses experimental results for both the offline and online components. Chapter Five concludes the thesis, summarises its contributions, discusses limitations, and outlines directions for future research.

CHAPTER TWO

LITERATURE REVIEW

2.1 PURPOSE AND SCOPE OF THE LITERATURE REVIEW

This chapter reviews the literature relevant to per-instance sorting algorithm selection. The goal is not to enumerate every sorting study, but to build the theoretical and empirical foundation for using machine learning to choose among sorting algorithms based on structural features of numeric arrays.

2.2 ORGANIZATION OF THE CHAPTER

The chapter moves from sorting as an input-sensitive operation (Section 2.3) to the structural properties of input arrays (Section 2.4), formal algorithm selection (Section 2.5), machine-learning-based selection (Section 2.6), the feature set used in this thesis (Section 2.7), the supervised learning pipeline (Section 2.8), adaptive and learning-augmented sorting (Sections 2.9 and 2.10), real-world numeric domains and Formula 1 telemetry (Section 2.11), domain generalization and routed models (Section 2.12), online adaptation (Section 2.13), evaluation metrics (Section 2.14), and the identified research gap (Section 2.15).

2.3 SORTING ALGORITHMS AND INPUT-DEPENDENT PERFORMANCE

Sorting is a fundamental operation in computing, but the practical performance of sorting algorithms is not determined by input size alone. Algorithms with similar asymptotic complexity may behave very differently depending on the structural properties of the input array, such as existing order, duplicate density, value diversity, runs, and distribution shape. This input-dependent behaviour motivates per-instance algorithm selection, where the goal is to choose the most suitable sorting algorithm for each array rather than rely on a single fixed algorithm (Sundaramoorthy & Karunanidhi, 2025a).

Systematic evaluation of sorting algorithms puts into consideration various theoretical factors such as time complexity, space complexity, and stability, together with

empirical analysis across a range of data types, sizes, and distributions (Sundaramoorthy & Karunanidhi, 2025a).

2.3.1 COMPARISON-BASED AND NON-COMPARISON-BASED SORTING

Sorting algorithms are usually divided into two families based on how they establish order. Comparison-based algorithms rely only on pairwise comparisons and assume only that the elements admit a total order (Gill et al., n.d.). Non-comparison-based algorithms such as Counting Sort, Radix Sort, and Bucket Sort exploit internal key structure (digits, integer ranges) to place elements directly (Amini, 2026). Under restricted assumptions on the data domain, non-comparison methods can achieve linear time, but their applicability depends on those assumptions (Salihu et al., 2022).

This thesis focuses on general-purpose sorting algorithms that operate on arbitrary numeric arrays without domain-specific key structure. Non-comparison-based sorting remains relevant mainly as a point of comparison with hybrid systems such as Adaptive Hybrid Sort, which incorporates Counting Sort and Radix Sort. The table below, Table 2.1, summarizes the key differences between comparison-based and non-comparison-based sorting

Table 2.1: Comparison-based sorting vs non-comparison based sorting

Category	Examples	Main Assumption	Relevance to Thesis
Comparison-based sorting	Quicksort, heapsort, mergesort, timsort, introsort, shell sort	Elements can be compared using an ordering relation	Main algorithm portfolio
Non-comparison-based sorting	Counting sort, radix sort, bucket sort	Values have usable key/range/digit structure	Relevant mainly as comparison with Adaptive Hybrid Sort

2.3.2 LOWER BOUND OF COMPARISON-BASED SORTING

Any comparison-based sorting algorithm must distinguish among the $n!$ possible orderings of an input of size n . A decision-tree argument shows that this requires depth at least $\log_2(n!) = \Omega(n \log n)$, which gives the well-known $\Omega(n \log n)$ lower bound in the comparison model.

Several candidate algorithms attain $O(n \log n)$ in the average or worst case. However, asymptotic equivalence does not imply equivalent practical runtime. Algorithms within the same complexity class are separated by constant factors, cache behaviour, memory access patterns, and sensitivity to input structure. Even when every candidate is asymptotically optimal, empirical and feature-based selection therefore remains useful (Aly et al., 2025a).

2.3.4 QUICKSORT

Quicksort is a divide-and-conquer algorithm that partitions arrays around a pivot. Depending on pivot strategy and input order, its average-case time complexity is $O(n \log n)$ with low overhead, while its worst case is $O(n^2)$ (Sundaramoorthy & Karunanidhi, 2025b).

Due to their low overhead and strong average-case behaviour quicksort and quicksort-derived strategies have strongly influenced practical sorting implementations. Many production sorting implementations do not use plain quicksort: they use hybrid variants such as introsort to bound worst-case behaviour (Rob et al., 2026a).

Quicksort is highly input-sensitive: pivot quality, the presence of duplicates, and partial order in the input all affect its empirical runtime, making it an important candidate in a sorting portfolio for per-instance selection (Mohammadagha & Student, 2025).

2.3.5 INTROSORT

Introsort is a hybrid algorithm that begins as quicksort, switches to insertion sort for small sub-arrays, and falls back to heapsort once recursion depth exceeds $2\lfloor \log n \rfloor$. This combination preserves the average-case speed of quicksort while bounding the worst case at $O(n \log n)$.

Empirical studies report that with low memory overhead introsort is consistently fast and predictable across input distributions. It is therefore a robust general-purpose candidate in this thesis because it is the basis for the C++ Standard Template Library sort (Akobre í mí et al., 2025).

2.3.6 HEAPSORT

Repeated heap construction and extraction to sort in place is used by Heapsort, guaranteeing $O(n \log n)$ time and $O(1)$ auxiliary space in every case. Due to weak cache locality, it is typically slower than quicksort and mergesort on random inputs but its runtime is highly predictable (Aly et al., 2025b).

In settings where predictable worst-case behaviour and low auxiliary memory are important, heapsort is relevant. In modern systems it most often appears as the worst-case fallback inside introsort rather than as a standalone choice (Akobre í mí et al., 2025).

2.3.7 MERGESORT

Sorting is done by mergesort by recursively splitting the array, sorting each half, and merging the sorted halves. It guarantees $O(n \log n)$ time, is stable, and uses $O(n)$ auxiliary memory (Rob et al., 2026b).

Mergesort has good cache behaviour due to its sequential memory access pattern. Its stability and predictable scaling make it a useful candidate where the relative order of equal keys must be preserved or where reproducible runtime is more important than absolute speed (Aly et al., 2025b).

2.3.8 TIMSORT AND RUN-ADAPTIVE SORTING

Timsort was originally developed for the Python runtime and was later adopted by Java and Android as the default sort for object arrays. It detects maximal monotonic sub-sequences (runs) and merges them using insertion sort and mergesort, with a galloping merge routine for runs that contain long stretches from a single side.

Studies of timsort’s galloping behaviour show that it sorts arrays with two distinct values in linear time, although it does not avoid $\Theta(n \log n)$ comparisons on arrays with three or more distinct values (Ghasemi et al., 2024). Powersort, which is now the default list-sorting algorithm in Python 3.11, generalises timsort’s merge policy and improves run-adaptive performance. A 4-way variant of Powersort has been reported to give 15–20% speed-up over standard Powersort by reducing memory transfer costs.

Run-adaptive algorithms are particularly relevant in this thesis because many real-world numeric arrays (for example, Formula 1 telemetry channels) contain long monotonic or near-monotonic regions.

2.3.9 SHELL SORT

Shell sort generalises insertion sort using diminishing-increment gap sequences. Its time complexity depends on the gap sequence and is gap-dependent in both the average and worst case.

Shell sort remains competitive on small or nearly sorted arrays where minimal overhead and in-place operation matter more than asymptotic optimality. It is included as a candidate in the sorting portfolio used in this thesis because it is empirically strong on inputs with low disorder (Sundaramoorthy & Karunanidhi, 2025b).

Table 2.2: Comparative Summary of Candidate Sorting Algorithms

Algorithm	Average Time	Worst Time	Stable	Extra Memory	Adaptive Behaviour	Relevance to Thesis
Quicksort	$O(n \log n)$	$O(n^2)$	No	Low	Pivot-dependent	Often fast, input-sensitive
Introsort	$O(n \log n)$	$O(n \log n)$	No	Low	Hybrid fallback	Robust general-purpose candidate
Heapsort	$O(n \log n)$	$O(n \log n)$	No	Low	Limited	Predictable worst-case behaviour
Mergesort	$O(n \log n)$	$O(n \log n)$	Yes	$O(n)$	Limited	Stable and predictable
Timsort	$O(n \log n)$	$O(n \log n)$	Yes	$O(n)$	Strong run adaptivity	Useful for partially sorted arrays
Shell sort	Gap-dependent	Gap-dependent	No	Low	Gap-dependent	Useful empirical candidate

Taken together, these algorithms summarised in Table 2.2 above span several distinct trade-offs in time, stability, memory, and adaptivity. No single algorithm is uniformly best, which is the empirical condition that makes per-instance algorithm selection potentially useful (Xu et al., 2024).

2.4 INPUT STRUCTURE AND SORTING PERFORMANCE

The sorting literature consistently finds that relative performance rankings are very sensitive to input attributes and that no single algorithm dominates across all input structures (Sundaramoorthy & Karunanidhi, 2025b). This input sensitivity is the empirical justification for per-instance algorithm selection. The remainder of this section discusses the input structures most relevant to the feature set used in this thesis.

2.4.1 ARRAY SIZE AND RUNTIME SCALING

The most obvious factor affecting runtime is array size, captured by the features `n_elements` and `length_norm`. All sorting algorithms slow down as input grows, and constant-factor differences become important at large n . Size alone does not determine which algorithm is fastest: two arrays of equal length can have very different runtimes depending on their internal structure (Akobre í mí et al., 2025).

2.4.2 SORTEDNESS AND DISORDER

Inputs are rarely fully random in practice. Real arrays may be sorted, reverse sorted, nearly sorted, or partially sorted. The features `adj_sorted_ratio` (local sortedness) and `inversion_ratio` (global disorder) capture how far an array is from a sorted ordering. Adaptive algorithms such as timsort exploit low disorder, while algorithms such as heapsort are largely insensitive to it (Raman et al., 2023).

2.4.3 RUNS AND NATURAL ORDER

A maximal consecutive ordered region within an array a run is. The `runs_ratio` and `longest_run_ratio` features summarise how much natural order is present. Run structure is the basis of timsort and Powersort, so arrays with many long runs tend to favour run-adaptive sorting (Jugé, 2024).

2.4.4 DUPLICATES AND REPEATED VALUES

Partitioning and merging behaviour are affected by duplicates. Highly duplicate or low-cardinality arrays (such as repeated telemetry samples) can degrade distribution-based sorting because of bucket overflow. The `duplicate_ratio`, `top1_freq_ratio`, and `top5_freq_ratio` features characterise this concentration of values (Akobre í mí et al., 2025).

2.4.5 ENTROPY AND VALUE DIVERSITY

Repetitive arrays are distinguished from diverse ones using entropy. The feature `entropy_ratio` is low for arrays that contain many repeated values and high for arrays with many distinct values. This complements duplicate counts by reflecting the overall diversity of the value distribution (Smith-Miles & Muñoz, 2023).

2.4.6 DISTRIBUTION, SPREAD, SKEWNESS, AND OUTLIERS

Real-world numeric arrays vary in spread and shape. Financial and cryptocurrency data have heavy tails, seismic data contain spikes, and weather data are usually smooth. `Dispersion_ratio`, `iqr_norm`, and `mad_norm` capture spread. Shape is captured by `skewness_t` and `kurtosis_excess_t`. `Mean_abs_diff_norm` and `outlier_ratio` capture local variation and outliers. Together these features describe the numeric profile of an array beyond simple ordering statistics (Uddin & Lu, 2024).

Table 2.3: Mapping Input Structures to Thesis Features

Input Structure	Thesis Features	Why It Matters for Sorting
Size	<code>n_elements</code> , <code>length_norm</code>	Affects runtime scale
Local sortedness	<code>adj_sorted_ratio</code>	May favour adaptive sorting
Global disorder	<code>inversion_ratio</code>	Measures distance from sorted order
Runs	<code>runs_ratio</code> , <code>longest_run_ratio</code>	Important for Timsort-like behaviour
Duplicates	<code>duplicate_ratio</code> , <code>top1_freq_ratio</code> , <code>top5_freq_ratio</code>	Affects comparison and partition behaviour
Diversity	<code>entropy_ratio</code>	Distinguishes repetitive and varied arrays

Spread	dispersion_ratio, iqr_norm, mad_norm	Captures numeric distribution
Shape	skewness_t, kurtosis_excess_t	Captures asymmetry and tails
Local variation / outliers	mean_abs_diff_norm, outlier_ratio	Useful for time-series and real-world numeric data

This mapping, shown in Table 2.3, shows how each feature used in the thesis is grounded in a known structural property of input arrays. Each property has been independently linked in the literature to differential sorting performance, which is what makes a feature-based selector plausible.

2.5 THE ALGORITHM SELECTION PROBLEM

The task of choosing, for a given problem instance, the best algorithm from a set of candidates with respect to a performance criterion such as runtime, solution quality, or resource use is called The Algorithm Selection Problem (ASP). ASP was formalised by Rice in 1976 and remains highly relevant given the diversity of solvers, parameter settings, and hardware in modern computing. It has been applied to AutoML, black-box continuous optimisation, and combinatorial problems such as SAT and TSP (Tornede et al., n.d.).

Per-instance algorithm selection is motivated in part by the implications of the No Free Lunch theorem: no single algorithm dominates across all instance distributions, so selecting per instance can outperform any fixed choice. The main practical obstacles include feature engineering cost, portfolio design, generalisation to unseen instances, and the overhead of running the selector itself. Benchmarks such as ASlib and BBOB are commonly used to evaluate selection methods (Goldblum et al., 2023).

2.5.1 RICE’S ALGORITHM SELECTION FRAMEWORK

The foundational model for ASP is Rice’s 1976 framework. It defines four interrelated spaces:

- Problem Space (P): the set of all problem instances.
- Algorithm Space (A): the portfolio of candidate algorithms or solver configurations.

- Feature Space (F): measurable characteristics of problem instances, including structural, landscape, or statistical features.
- Performance Space (Y): quantitative metrics such as runtime, solution quality, penalised average runtime (PAR10), or success rates.

The goal is to learn, from training data, a selection mapping $S: F \rightarrow A$ (or to a distribution over A) that maximises expected performance. The framework supports both offline training and online adaptation and provides a common vocabulary across domains (Tornede et al., n.d.).

2.5.2 MAPPING RICE’S FRAMEWORK TO SORTING ALGORITHM SELECTION

In this thesis, the problem includes: numeric arrays, the algorithm space is the set of candidate sorting algorithms, the feature space consisting of structural array descriptors, and the performance measure is empirical sorting runtime. The selection mapping is learned using supervised machine learning, predicting the fastest sorting algorithm for each array which is the goal.

Table 2.4: Translation of Rice Framework Component to Usage in this Thesis

Rice Framework Component	General Meaning	Meaning in This Thesis
Problem space	Set of problem instances	Numeric arrays
Feature space	Measurable instance characteristics	Structural array features
Algorithm space	Candidate algorithms or solvers	Quicksort, introsort, heapsort, mergesort, shell sort, timsort
Performance space	Runtime, cost, quality, resource use	Sorting runtime
Selection mapping	Function from features to algorithm	ML model, mainly XGBoost

This mapping shown in Table 2.4 is the backbone of the thesis. Every later section can be read as an instantiation of one of these spaces.

2.5.3 PER-INSTANCE ALGORITHM SELECTION

Per-instance algorithm selection learns a predictive model that ranks or recommends the best algorithm in a portfolio based on features extracted from a new instance. Common formulations include classification (predict the winner), regression (predict performance), clustering, and learning to rank. Per-instance selection exploits performance complementarity by adapting to each instance, in contrast to per-set methods that pick one algorithm for an entire instance distribution (Kerschke et al., 2018).

Recent work has extended per-instance selection in several directions: online selection that incorporates information gathered during execution, uncertainty-aware models that improve reliability across runs, and application-specific selectors for problems such as subgraph isomorphism and recommender systems. Reported improvements over single best solvers are often in the 10–50% range on standard benchmarks (Shavit & Hoos, 2024).

2.5.4 ALGORITHM PORTFOLIOS AND SELECTION MAPPINGS

A set of algorithms with complementary strengths is called a portfolio. Selection mappings choose, schedule, or combine portfolio members for a given instance. When no single algorithm dominates selection is useful, but a portfolio that is too large can also raise selection cost and training data requirements (Pezo et al., 2025).

The algorithm portfolio is not automatically generated from hundreds of solver variants in this thesis. It is manually defined using well-known sorting algorithms with different practical characteristics. The usefulness of this portfolio depends on performance complementarity: if different sorting algorithms are fastest for different types of arrays, then a feature-based selector can improve over the single best fixed algorithm.

Synthesis: The formal vocabulary in which this thesis is written is provided by algorithm selection. Sorting is treated as one application domain of the Rice framework, with arrays as instances, structural features as inputs, and runtime as the performance criterion.

2.6 MACHINE LEARNING-BASED SOLVER AND ALGORITHM SELECTION

Machine-learning-based solver selection automates decisions that previously required domain experts. It learns a mapping from instance features to the best algorithm or to predicted performance instead of asking which algorithm to use. The most influential developments come from the SAT-solver and combinatorial-optimisation communities, but the methodology generalises to any portfolio with measurable instance features (Tornede et al., n.d.).

2.6.1 SATZILLA AND FEATURE-BASED SOLVER SELECTION

A portfolio-based SAT solver that uses empirical hardness models to choose among component solvers on a per-instance basis is called SATzilla (Xu, Hutter, Hoos, and Leyton-Brown, 2008). Given a distribution of SAT instances and a set of component solvers, SATzilla predicts solver runtime from instance features and selects the predicted best solver, optimising objectives such as mean runtime or competition score. Its competition results established feature-driven solver selection as a research standard, and many later systems are evaluated against it (Xu et al., 2024).

More recent work has updated SATzilla’s feature pipeline (for example, by replacing SATElite with SBVA preprocessing) and explored graph-based alternatives such as GraSS, which represents instances as tripartite graphs to capture latent structure that hand-engineered features miss. These extensions show that feature engineering remains a moving target as benchmarks evolve, but they do not change SATzilla’s central role as the canonical example of ML-based per-instance selection (Zhang et al., 2024).

Table 5: Comparison of the SATzilla Solver Selection Framework with This Thesis

Aspect	SATzilla	This Thesis
Problem instance	SAT formula	Numeric array
Candidate methods	SAT solvers	Sorting algorithms
Features	SAT instance features	Array structural features
Target	Best solver or runtime	Fastest sorting algorithm

Learning role	Empirical hardness/performance modelling	XGBoost-based selection
Evaluation	Runtime, solved instances, competition score	Runtime, regret, SBS, VBS, gap closed

Table 2.5 makes the analogy explicit: SATzilla is to SAT solvers what the selector in this thesis is to sorting algorithms.

2.6.2 EMPIRICAL PERFORMANCE MODELS

Regression models that predict algorithm performance, usually runtime, from instance features and (optionally) configuration parameters are called Empirical Performance Models (EPMs). Continuous performance estimates for every algorithm in a portfolio are produced by EPMs, so the selector can choose the algorithm with the lowest predicted runtime (Hutter et al., 2014). They are used for algorithm analysis, portfolio selection, and surrogate-based algorithm configuration.

An EPM-based formulation would predict runtime for each candidate sorting algorithm and choose the one with the lowest predicted runtime in the context of this thesis. The thesis mainly uses classification, but EPMs remain a natural alternative formulation that is sensitive to runtime magnitude rather than only to the identity of the winner.

2.6.3 CLASSIFICATION-BASED SELECTION

Algorithm choice is treated as a discrete labelling problem using classification-based selection: the model receives instance features and predicts the index of the best algorithm. This is computationally efficient at deployment because a single forward pass produces the choice. It is also conceptually simple and matches the structure of many supervised learning libraries, including XGBoost (Díaz de León-Hicks et al., 2023).

2.6.4 RUNTIME REGRESSION AND RANKING-BASED SELECTION

Regression-based selection predicts runtime per algorithm and selects the minimum. It carries more information than classification but is more sensitive to timing noise and

label scaling. Ranking-based methods predict an order over algorithms and are useful when fallbacks are needed, for example in anytime scheduling (Pezo et al., 2025).

Table 2.6: Comparison of Algorithm Selection Approaches: Classification, Regression, and Ranking

Approach	Prediction Output	Selection Rule	Advantage	Limitation
Classification	Best algorithm label	Choose predicted class	Simple and direct	Ignores magnitude of runtime difference
Runtime regression / EPM	Runtime per algorithm	Choose lowest predicted runtime	Runtime-aware	Sensitive to timing noise
Ranking	Algorithm order	Choose top-ranked algorithm	Useful for fallback	More complex

Table 2.6 above summarises the key characteristics of algorithm portfolios in the context of per-instance algorithm selection, highlighting their advantages, design considerations, and relevance to this thesis.

2.6.5 RELEVANCE TO SORTING ALGORITHM SELECTION

This thesis mainly follows the classification formulation, because each array is assigned the label of the empirically fastest sorting algorithm. However, evaluation cannot rely only on classification accuracy: selecting a non-fastest algorithm may have little practical cost when runtimes are close, and may have large cost when they are far apart. Runtime-sensitive metrics such as regret and gap closed (Section 2.13) are therefore necessary complements to accuracy (Renau & Hart, 2024a).

2.7 FEATURE-BASED INSTANCE CHARACTERISATION

2.7.1 MOTIVATION FOR STRUCTURAL ARRAY FEATURES

Each input array must be encoded as a fixed-length numerical vector according to the sorting algorithm selector developed in this thesis before being processed by a supervised learning model. A model that expects inputs of uniform dimensionality

cannot simply compare an array of ten integers with an array of ten million integers since arrays are variable-length sequences, which makes this condition non-trivial. In addition to preventing a classifier from generalising across lengths, feeding it raw array contents would compel the model to learn sorting-relevant patterns from individual values whose absolute magnitudes lack structural information about algorithm performance. A classifier cannot determine whether Timsort or Quicksort will be faster on the containing array based on a value of 42 in position 7.

The approach used in the literature on algorithm selection and meta-learning is to summarise each instance's structural qualities using a fixed-length vector of hand-engineered or learnt features (Smith-Miles & Muñoz, 2023). The degree of existing order, the run structure, the frequency of repeated values, the statistical shape of the value distribution, and the presence of outliers are the structurally relevant properties in the sorting context that are known from the sorting literature to differentiate algorithm behaviour. The feature vector is an understandable, condensed representation of each array instance as each feature in the thesis feature set is based on one or more of these characteristics.

The main classifier in this thesis, XGBoost, is a gradient-boosted decision tree ensemble that has been shown to perform better than deep learning models on tabular data of the size commonly found in method selection studies (Grinsztajn et al., 2022a). The selection of fixed-length tabular features is supported by its architectural constraints, which mandate that each training row be a vector of scalar values aligned across instances. In a field where training data is produced by empirical benchmarking and feature interpretability is part of the scientific contribution, deep learning alternatives that can process variable-length sequences such as recurrent or transformer-based architectures would circumvent this requirement but come with significantly higher data requirements, greater computational costs, and reduced interpretability (Shwartz-Ziv & Armon, 2022).

Regardless of the labels used to train the classifier, each feature in the thesis feature set must have a verifiable connection to a sorting-relevant structural aspect of the input array. This concept guarantees that the feature engineering process is transparent and

that sorting theory may be used to describe the final model. Each feature group is justified in those terms throughout the rest of this section.

2.7.2 SIZE FEATURES

The two size features are `length_norm`, a normalised version that transfers array length to a limited interval to avoid scale dominance in the feature vector, and `n_elements`, the raw count of elements in the array. The most basic aspect influencing sorting runtime is array size: all sorting algorithms slow down as input increases, and the practical importance of method selection rises with n as the constant-factor disparities between algorithms build up over more elements.

There are differences in runtime scaling amongst algorithms. In reality, Quicksort is quicker than Mergesort for moderately big arrays due to its predicted $O(n \log n)$ runtime with very low constant factor. However, at large n , where worst-case guarantees are more important than average-case speed, Heapsort's guaranteed $O(n \log n)$ becomes significant. For large arrays, Timsort's overhead from run detection is insignificant, but for very tiny arrays, where insertion sort is faster, its advantages could be outweighed. Because of these scaling correlations, algorithm selection can already be guided by size alone before any ordering aspects are looked at.

Size, however, is clearly insufficient for precise selection. Depending on whether one is almost sorted (favourable for Timsort) or has a lot of duplicates (favourable for three-way partitioning), two arrays of the same length may have entirely different runtimes. Input size and input structure must be taken into account when determining actual sorting performance because neither of them can accurately predict performance differences on its own (Sundaramoorthy & Karunanidhi, 2025). This drives the multi-feature strategy used in this thesis, which supplements the structural features discussed in the next subsections with size features as necessary but insufficient predictors.

2.7.3 SORTEDNESS AND DISORDER FEATURES

The degree of existing order in an array is characterised by two features: `inversion_ratio`, which normalises the total inversion count to the interval $[0, 1]$ (global disorder), and `adj_sorted_ratio`, which measures the proportion of adjacent element pairs that are already in non-decreasing order (local sortedness). These features capture

complementary aspects of presortedness: `inversion_ratio` represents cumulative global disorder, while `adj_sorted_ratio` is sensitive to local ordering structure.

The traditional formal metric of presortedness is inversion count. The inversion count $\text{Inv}(X)$ for an array X of length n is the number of pairs (i, j) where $i < j$ and $X[i] > X[j]$. $\text{Inv}(X) = 0$ for a fully sorted array and $n(n-1)/2$ for a reverse-sorted array. By dividing by this maximum, the normalised `inversion_ratio` yields a number in $[0, 1]$ that is consistent across arrays of various lengths. Inversion count is one of the most theoretically supported measurements of how much an array deviates from sorted order, along with run count and maximum displacement. Mannila (1985) established the foundational axiomatic framework for presortedness measures, confirming inversion count as a canonical metric of disorder.

There is a clear and established link to adaptive sorting performance. Bai and Coester (2023), in a NeurIPS paper on learning-augmented sorting, show that comparison complexity degrades as a smooth function of prediction error, which is closely related to the inversion structure of the input, confirming that presortedness directly shapes algorithmic cost.

2.7.4 RUN-BASED FEATURES

An array's natural block structure is characterised by run-based properties, which break it down into maximal successive monotonic sub-sequences. The two run features are `longest_run_ratio`, which is defined as the length of the longest ascending run divided by n (normalised longest run length), and `runs_ratio`, which is defined as the number of ascending runs divided by n (normalised run count). Many short runs (small block order) are indicated by a `runs_ratio` near 1, whereas few lengthy runs (strong block order) are indicated by a `runs_ratio` near 0. One sizable section of the array is already sorted if the `longest_run_ratio` is high.

The main structural feature that Timsort and Powersort are specifically made to take advantage of is run structure. In order to achieve near-optimal merge cost as a function of the entropy of the run-length distribution, Powersort replaces Timsort's heuristic

merge policy with a nearly optimal one derived from the theory of alphabetic trees (Gelling et al., 2023). Timsort finds maximal natural runs during a single left-to-right pass and merges them using a galloping merge policy. Run structure is the theoretically correct foundation for adaptive merge-sorting, as demonstrated by Jugé's (2024) proof of the optimality of Adaptive ShiversSort within the class of k -aware natural merge-sort algorithms: any algorithm that fails to detect runs must pay an asymptotic penalty in comparison to one that does.

Low `runs_ratio` and high `longest_run_ratio` are powerful positive indicators of Timsort and Powersort-style selection for the thesis selector. The run-based advantage of Timsort collapses toward its worst-case $O(n \log n)$ behaviour with a high constant factor, making Quicksort or Introsort more competitive. In contrast, arrays with `runs_ratio` close to 1 and `longest_run_ratio` close to 0 are almost random in block structure. A decision-tree classifier like XGBoost can capture this type of conditional performance complementarity with its splitting rules.

2.7.5 DUPLICATE AND FREQUENCY FEATURES

The density and distribution of repeated values are characterised by three features: `top1_freq_ratio` (frequency of the most common value divided by n), `top5_freq_ratio` (combined frequency of the five most common values divided by n), and `duplicate_ratio` (percentage of non-unique elements). When combined, these characteristics characterise the concentration of the most common values as well as the general frequency of repeat.

Partitioning algorithms are directly and clearly impacted by duplicate density. Because the partitioning step may result in very uneven sub-arrays when many items are equal to the pivot, standard two-way Quicksort degrades toward $O(n^2)$ on arrays with numerous duplicates. This situation is best handled by three-way Quicksort (Dutch National Flag partitioning), which groups all elements identical to the pivot in a single pass and removes their contribution to subsequent recursive work. Therefore, a high `duplicate_ratio` is a good indicator that three-way partitioning variations will perform better on the given array than two-way Quicksort and regular Mergesort.

The concentration dimension of the duplicate structure is addressed by the `top1_freq_ratio` and `top5_freq_ratio` features. An array with 30 separate values that each occur twice is fundamentally different from an array where one value makes up 80% of the elements. The distribution of key repetition is a crucial axis of variation in database sort performance, as shown by (Raman et al., 2023) BoDS benchmark, which measures sortedness as a two-dimensional quantity including the (K, L)-sortedness metric. Systems that are unable to exploit repeated keys exhibit order-of-magnitude degradation in comparison to those that can. Together, the three duplication and frequency characteristics in the thesis feature set give the selector the information it needs to recognise such arrays and direct them to algorithms that use suitable duplicate-handling techniques.

2.7.6 ENTROPY FEATURE

The normalised Shannon entropy of the array's empirical value distribution is used to calculate the entropy feature, `entropy_ratio`. The definition of Shannon entropy H for an array of n elements taken from a value alphabet of size k is $H = -\sum p_i \log_2(p_i)$, where p_i is the empirical probability of value i . The feature is bounded in $[0, 1]$ by the normalised form $\text{entropy_ratio} = H / \log_2(n)$, where 0 denotes a constant array (all elements identical) and 1 denotes a uniform distribution over n different values.

Through offering a single scalar summary of the total diversity of the value distribution, entropy enhances the duplicate features. Entropy integrates data throughout the whole value distribution, whereas `duplicate_ratio` catches the percentage of repeated items and `top1_freq_ratio` records the dominance of the single most common value. Even while two arrays have the same `duplicate_ratio`, their entropy values can differ significantly: a high entropy array has a uniform distribution of duplicates across many values, whereas a low entropy array has almost all repetition accounted for by a small number of entries. In a systematic review of entropy applications in machine learning (Sepúlveda-Fontaine & Amigó, 2024), confirm that entropy features enhance classifier performance precisely in settings where the value distribution shapes the computational complexity of the target task. (Garcia-González et al., 2023) show that Shannon entropy-derived features give classifiers distributional information that cannot be recovered from simpler statistical summaries.

In addition to the sortedness features, low `entropy_ratio` values are useful for choosing sorting algorithms because they demonstrate that the value space is sparse independent of element ordering. This is pertinent to algorithms that deal with value structure, including non-comparison-based techniques (Counting Sort, Radix Sort), which are part of the Adaptive Hybrid Sort comparison but not the primary candidate portfolio. In the comparison-based portfolio, low `entropy_ratio` and high `duplicate_ratio` frequently co-occur, and when the two features are combined, the classifier receives a stronger signal than when either feature is used alone.

2.7.7 DISPERSION AND ROBUST SPREAD FEATURES

Three characteristics of the element value distribution are `dispersion_ratio` (normalised range), `iqr_norm` (normalised interquartile range), and `mad_norm` (normalised median absolute deviation). Instead of using the standard deviation alone, three complementary robust estimators were chosen due to the statistical properties of real-world numerical arrays, which frequently show non-normal, heavy-tailed distributions. Each encapsulates a unique aspect of value dispersion.

The standard deviation, a widely used measure of spread that is prone to outliers and misrepresents the dispersion of the bulk of the array, can be greatly inflated by a single extreme figure. The interquartile range (IQR), which is immune to the tails, gets around this problem by estimating the spread of the middle 50% of the distribution. (Dallah & Sulieman, 2024) have empirically shown that IQR-normalized range statistics regularly outperform standard-deviation-based statistics in spotting outliers across heavy-tailed distributions (Laplace, logistic, Weibull). This has a direct bearing on the seismic and financial data that were utilised to evaluate the theory. As a result, the `iqr_norm` feature captures value spread in a way that is resistant to the tail behaviour that characterises many of the real-world array types used in this thesis.

The median of the absolute departures from the median is known as the median absolute deviation (MAD): $MAD = \text{median}(|X_i - \text{median}(X)|)$. It is the most resilient of the three spread features against outlier contamination since it has a breakdown point of 50%, which means that up to half of the observations can be substituted with arbitrary values without making the estimate excessively big. In an article on

confidence intervals for MADs that was published in Communications in Statistics (2024), (Clarke et al., 2024) affirm that MAD offers trustworthy dispersion estimates in a variety of skewed distributions where standard deviation falls short. For time-series arrays from fields like seismic monitoring, where impulsive noise might occasionally create high values that would skew variance-based spread estimates, the `mad_norm` feature is very pertinent.

`Dispersion_ratio`, `iqr_norm`, and `mad_norm` together provide the selection with three separate perspectives of value distribution at various robustness levels. Beyond the above theoretical justification (Abril-Pla et al., 2023a) confirm that spread features typically have a statistically significant impact on the performance of tabular classifiers and that their significance varies across algorithm types, making their inclusion in the feature set empirically justified.

2.7.8 SHAPE AND OUTLIER FEATURES

`Skewness_t` (bias-corrected sample skewness), `kurtosis_excess_t` (bias-corrected excess kurtosis), `mean_abs_diff_norm` (normalised mean absolute difference between adjacent elements), and `outlier_ratio` (proportion of elements classified as outliers by the IQR rule) are the four features that describe the array's local variability and value distribution shape. These characteristics are especially crucial for characterising real-world numerical arrays from fields that are included in the thesis evaluation package, such as financial markets, cryptocurrency price series, weather data, and seismic readings.

The asymmetry of the value distribution is captured by skewness: negative skewness denotes the opposite, while positive skewness denotes a right-extended tail with most values concentrated at the lower end with sporadic very big values. The literature on higher-order moments in financial data has widely demonstrated the systematic left- or right-skewing of financial and cryptocurrency return series according on market conditions (Karagiorgis et al., 2024) Skewed arrays present asymmetric partitioning problems for sorting algorithm selection: a Quicksort pivot selected from a right-skewed distribution will often be far from the median, resulting in uneven sub-arrays and worsening toward the worst case. Therefore, pivot techniques that are resistant to

distributional asymmetry, such median-of-three or median-of-medians selection, have a greater relative advantage when skewness_t values are high.

Excess kurtosis quantifies the distribution's tail weight in relation to a normal distribution: negative excess kurtosis (platykurtosis) denotes lighter tails, while positive excess kurtosis (leptokurtosis) suggests heavier tails with more extreme values than a Gaussian would predict. Strongly positive excess kurtosis in cryptocurrency data is well-documented; numerous studies have confirmed that kurtosis for daily return series is at least ten times higher than that of normal distributions (Karagiorgis et al., 2024). According to (Abril-Pla et al., 2023b) kurtosis is a crucial meta-feature for algorithm selection models since it has a statistically significant detrimental impact on the performance of non-tree-based classifiers. High kurtosis_excess_t for sorting denotes arrays with a lot of near-median values and sporadic extreme outliers, which makes pivot-selection difficult and may prefer methods like Heapsort or Introsort that have deterministic worst-case bounds.

Local variation is captured by the mean_abs_diff_norm feature, which normalises the average magnitude of the difference between neighbouring elements by n . In contrast to global spread, an array may exhibit low dispersion but high local variation (a quickly oscillating sequence) or high dispersion but low local variation (a smooth monotone sequence). High local variation is a helpful addition to the run-based features since it is linked to high inversion density and numerous short runs. Local variation measures the level of measurement channel volatility for time-series data like Formula 1 telemetry, which influences the number of natural runs Timsort can identify and combine.

The percentage of array elements beyond the IQR-based fence $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ is known as the outlier_ratio feature. In a paper on outlier detection using extreme value theory published in the Scandinavian Journal of Statistics, (Bhattacharya et al., 2023) show that the standard Tukey IQR fence accurately detects outliers under a variety of non-normal distributions, making outlier_ratio a dependable and comprehensible feature for describing the tail behaviour of array value distributions. Because the extreme values affect both the partitioning balance in

Quicksort-family algorithms and the merge cost of Mergesort-family algorithms when the runs containing outliers must be merged with numerous other runs, arrays with high outlier_ratio—such as seismic recordings with impulsive spikes or financial time series with crash events—present unique sorting challenges.

2.7.9 SUMMARY OF FEATURE GROUPS

Twenty-two characteristics total, arranged into six categories: size, sortedness and disorder, run structure, duplicates and frequency, entropy and diversity, dispersion and spread, and form and outliers—make up the whole feature set employed in this thesis. A combined reference for all features, their groupings, what each measures, and which sorting portfolio algorithms are most likely to be impacted by each feature group can be found in Table 2.6.5 below. This combined perspective shows that the feature set includes every fundamentally unique aspect of array instance characterisation that has been found to be performance-relevant by the sorting literature.

Table 2.6.5: *Consolidated Summary of All Thesis Features by Group*

Group	Feature(s)	What It Measures	Most Influenced Algorithms
Size	n_elements, length_norm	Array length and normalised length	All algorithms (runtime scale); constant-factor differences matter most at large n
Sortedness / Disorder	adj_sorted_ratio, inversion_ratio	Local ordering fraction; global inversion density	Timsort, Powersort (benefit from low disorder); Heapsort (insensitive)

Run Structure	runs_ratio, longest_run_ratio	Frequency and length of monotone runs	Timsort, (exploit runs); Quicksort (run-blind)
Duplicates / Frequency	duplicate_ratio, top1_freq_ratio, top5_freq_ratio	Overall and concentrated value repetition	Three-way Quicksort (handles duplicates); standard Quicksort (degrades)
Entropy / Diversity	entropy_ratio	Shannon entropy of value distribution	Indirectly affects all; low entropy co-signals high duplicate density
Dispersion / Spread	dispersion_ratio, iqr_norm, mad_norm	Value range, robust spread, and breakdown- point-robust spread	All algorithms; robust features most relevant for heavy-tailed real- world data
Shape / Outliers	skewness_t, kurtosis_excess_t, mean_abs_diff_norm, outlier_ratio	Distribution asymmetry, tail weight, local volatility, and extreme- value density	Quicksort (pivot quality); Introsort, Heapsort (worst-case bounds for heavy tails)

Since each group captures a unique structural dimension that cannot be fully recovered from the others, Table 2.6.5 shows that the feature set is not duplicated across groups. While they both relate to the current order, the sortedness and run characteristics capture it at different granularities (maximal block structure versus pairwise adjacency). While both the entropy feature and the duplicate feature are related to value repetition, they do not capture uniform diversity or concentrated repetition. The combination is informative for both well-behaved and heavy-tailed distributions since the three spread properties vary in how resilient they are to outliers. In addition to the

spread characteristics, the shape and outlier features provide information regarding distributional geometry.

The idea that algorithm performance in sorting is jointly determined by multiple structural properties and that the interaction effects between these properties are frequently non-linear justifies the multi-group design of the feature set. For instance, an array may have high `adj_sorted_ratio` (nearly sorted locally) but high `outlier_ratio` (with occasional extreme values that create long inversions). The net effect on Timsort performance depends on both properties jointly, and a decision-tree model like XGBoost is well suited to capture such interactions through its hierarchical splitting structure.

When combined, the 22 attributes offer a thorough, theoretically sound, and computationally effective instance representation that connects the six candidate algorithms' performance landscape with the structural characteristics of sorting inputs. The creation of this feature collection is what enables the thesis's per-instance machine-learning-based sorting algorithm selection feasible.

2.8 SUPERVISED LEARNING FOR SORTING ALGORITHM SELECTION

2.8.1 FORMULATING THE TASK AS SUPERVISED LEARNING

This thesis presents sorting algorithm selection as a supervised multi-class classification problem. The fixed-length structural feature vector mentioned in Section 2.6 serves as the model's input for each issue instance, which is a numeric array. The class label is the name of the sorting method that, according to benchmarks, is the fastest on that array. At inference time, the model is used to forecast the quickest technique for previously unseen arrays without re-running any of the candidate algorithms after being trained on a labelled dataset of (feature vector, label) pairs.

There are three reasons why the supervised formulation is suitable for this assignment. First, given a stable hardware environment, the mapping from structural array properties to algorithm performance is deterministic: an array with a certain run structure and inversion count will always prefer the same algorithm. A model trained

on a training dataset produced by benchmarking is predicted to generalise to fresh arrays selected from the same distribution because of its stability, which indicates that the dataset is a trustworthy representation of the underlying performance mapping. Second, there is no annotation subjectivity because the training labels for the fastest algorithm identities are derived from direct empirical measurement; the label is simply what the clock indicates. Third, supervised categorisation at inference time adds very little overhead in comparison to the sorting time itself, requiring only one forward pass through the trained model.

2.8.2 RUNTIME-BASED LABEL GENERATION

The sorting algorithm that achieves the lowest measured wall-clock runtime on that array is identified by the label given to each training instance. This section explains how near-ties are handled, how runtimes are monitored, and how the labels that are produced relate to the sorting portfolio's performance landscape.

Each of the six potential algorithms is applied separately to a new copy of each array. The copy is crucial because algorithms that sort in place would leave the array in a different state if they shared memory. Runtimes are measured using a high-resolution monotonic clock such as Python's `time.perf_counter` or C's `clock_gettime` with `CLOCK_MONOTONIC`, which is unaffected by system clock adjustments or time zone changes. Every algorithm is executed on the same array several times, and the representative runtime for that method on that array is determined by taking the median of the repeated measurements. Because it is resistant to sporadic outlier observations brought on by OS scheduling influence, cache warming effects, or transient background load surges, the median is favoured above the mean.

2.8.3 XGBOOST FOR TABULAR ALGORITHM SELECTION

The main classifier utilised in this thesis to train the mapping from structural array features to the fastest sorting technique is XGBoost (eXtreme Gradient Boosting), which was first introduced by Chen and Guestrin (2016). The gradient-boosted decision tree implementation XGBoost has continuously shown state-of-the-art performance on tabular supervised learning problems in a variety of fields, such as

algorithm selection, scientific computing, finance, and medicine (Grinsztajn et al., 2022b)

Five characteristics that are very important for choosing sorting algorithms support XGBoost's superiority over other classifiers. First, scalability and computational efficiency: XGBoost can quickly train on datasets generated by sorting benchmarks (tens of thousands of arrays) thanks to its regularised objective function, parallel tree construction, and cache-aware data access. This is crucial because the training dataset needs to be regenerated each time the candidate portfolio or hardware platform changes. Second, robust performance on tabular data: (Grinsztajn et al., 2022b)

demonstrate that gradient-boosted tree ensembles consistently outperform deep learning models on tabular classification tasks where the training set is medium-sized and the feature count is modest exactly the regime of this thesis in a NeurIPS benchmark study covering 45 datasets. Third, native handling of mixed-scale features: XGBoost's tree-based structure does not require feature normalisation or scaling, which is advantageous for the thesis feature set where features have heterogeneous units (counts, ratios, moments, entropy values) that would require careful scaling preprocessing for other methods. This is in contrast to distance-based classifiers like k-nearest neighbours or support vector machines.

Fourth, robustness to feature correlations: construction correlates several of the 22 thesis characteristics (e.g., `adj_sorted_ratio` and `inversion_ratio` both indicate sortedness from distinct perspectives). Because each split only chooses the locally most informative feature rather than merging all features at once, correlation has less of an impact on tree-based ensembles but lowers the coefficient estimates of linear models and inflates variance in kernel techniques.

2.8.4 FEATURE IMPORTANCE AND MODEL INTERPRETABILITY

Beyond predictive accuracy, interpretability is a crucial requirement for the thesis: the model should rely on features that are theoretically expected to drive sorting algorithm performance rather than on spurious correlations brought about by the training set's composition or the benchmarking process. This thesis makes use of two

complimentary interpretability tools: SHAP (SHapley Additive exPlanations) values and XGBoost's native feature significance scores.

Three metrics can be used to quantify the native feature importance of XGBoost: (i) gain, which is the average improvement in the objective function over all splits using a particular feature; (ii) frequency, also known as weight or count, which is the number of times a feature is used as a split across all trees; and (iii) cover, which is the average number of training instances affected by splits on a particular feature. Gain evaluates each feature's direct contribution to lowering classification error, making it the most informative of the three. A feature with high gain importance is one that regularly yields informative splits, which is precisely the characteristic that sorting-relevant structural features should possess if the model is operating as intended.

2.8.5 TIMING NOISE AND BENCHMARKING RELIABILITY

The dependability of the underlying timing measurements determines the quality of the runtime labels that power the supervised learning pipeline. When an algorithm's observed duration on a particular array differs significantly from its actual expected runtime due to timing noise, the label assigned to that array may be inaccurate, adding to label noise that reduces classifier accuracy. The main causes of temporal noise in sorting benchmarks are reviewed in this part along with the methods employed in this thesis to lessen their impact.

Several sources of temporal variability are introduced in contemporary computing settings. Operating system schedulers can incorporate arbitrary delays into the measured runtime by preempting a sorting process in the middle of its execution to execute another process. The number of concurrent processes, the scheduling policy, and system load all affect the likelihood and size of these preemptions. CPU frequency scaling: In order to handle heat or power constraints, dynamic voltage and frequency scaling (DVFS) may lower CPU clock frequency during benchmarks, hence inflating recorded runtimes in comparison to peak performance. Cache state: runtimes are dependent on execution history since the contents of the CPU cache at the beginning of a sorting run depend on what was executed just before. Just-in-time compilation: When hot spots are found in managed-runtime environments like the Java Virtual

Machine (JVM), code is first interpreted and then compiled to native code. Robust summary statistics in conjunction with repeated measurements are the typical mitigation for these causes of variability. The thesis protocol runs each algorithm on each array k times (where k is empirically determined to produce a coefficient of variation below a specified threshold) and uses the median of the resulting measurements as the representative runtime instead of depending on a single runtime measurement per algorithm per array. Because it is more resilient to the sporadic extreme outlier brought on by OS scheduling preemption or cache miss effects, the median is favoured above the mean. The median-based stability criterion is useful for identifying and eliminating unstable runs while minimising the overall benchmarking budget. Prior taking measurements, the initial iterations of each algorithm are discarded in order to address warm-up. This lessens the impact of cold-start effects by enabling the CPU branch predictor, data caches, and instruction caches to achieve a stable state for the benchmarked code. Warm-up also enables profile-guided compiler optimisations for compiled languages to finish before measurement starts.

The difference between the overhead of the selector and the runtime of the sorting algorithm is another factor to take into account. To ascertain whether the selector truly saves time overall on a given array, the feature extraction phase itself requires computation time, which must be factored into the full-system evaluation. However, as the labels are meant to represent the intrinsic performance of the sorting algorithms rather than the pipeline overhead, the feature extraction time is not included in the runtime measurements for the purposes of label generation and classifier training.

Timing noise has the practical effect of preventing classification accuracy from serving as the main metric in the assessment framework. In practice, an incorrect forecast that routes an array to an algorithm that is 0.001 milliseconds slower than the best option is insignificant, whereas an incorrect prediction that routes the array to an algorithm that is ten times slower can be important. This encourages the use of runtime-sensitive metrics as key evaluation metrics, such as gap closed in relation to the Virtual Best Solver and per-instance regret, with classification accuracy serving as a supplementary descriptive statistic.

2.9 ADAPTIVE, HYBRID, AND LEARNED SORTING

2.9.1 ADAPTIVE SORTING

Adaptive sorting algorithms exploit structural properties of the input, such as existing runs or near-sortedness, to perform less work than the worst-case bound. Run-adaptive merge sorting is the most prominent example. Studies on adaptive ShiversSort show that it is optimal within the class of k -aware merge-sort algorithms up to a small additive linear term in merge cost. This supports the broader view that run structure is an important factor in sorting performance (Jugé, 2024).

Powersort and its multiway extensions further reduce memory transfer costs while preserving near-optimal run-adaptive guarantees. A 4-way Powersort variant has been reported to achieve 15–20% speed-up over the standard two-way version (Gelling et al., 2023).

2.9.2 HYBRID SORTING AND INTROSORT

Hybrid sorting combines several strategies inside a single algorithm. The canonical example is introsort, which begins as quicksort and falls back to heapsort when recursion depth grows too large, with insertion sort used for small sub-arrays. Hybrid algorithms aim for the average-case speed of one strategy and the worst-case guarantee of another (Wiredu, Akobre, et al., 2025).

2.9.3 ADAPTIVE HYBRID SORT

Adaptive Hybrid Sort extends the hybrid idea by using lightweight learned rules to choose among internal strategies, including non-comparison sorts such as Counting Sort and Radix Sort. Reported gradient-boosted classifiers for this kind of internal selection achieve high accuracy on test cases drawn from controlled distributions. Adaptive Hybrid Sort is the closest related work, but it modifies a single sorting routine internally rather than selecting among independent algorithms (Díaz de León-Hicks et al., 2023).

Table 2.7: Comparison of Adaptive Sorting, Hybrid Sorting, and ML-Based Algorithm Selection

Concept	Meaning			Relation to Thesis
Adaptive sorting	One	algorithm	changes behaviour internally based on input	Related background
Hybrid sorting	One	algorithm	combines multiple strategies	Related background
ML-based algorithm selection	A	model	chooses among algorithms before execution	Main thesis direction

Table 2.7 above compares three related approaches to improving sorting performance: adaptive sorting, hybrid sorting, and machine-learning-based algorithm selection that helps in clarifying how the present thesis differs from existing adaptive and hybrid techniques.

Synthesis: most adaptive and hybrid sorting research focuses on improving the behaviour of a specific sorting algorithm. This thesis takes a different approach by using input features to select among several complete sorting algorithms before sorting begins.

2.10 PREDICTION-BASED AND LEARNING-AUGMENTED SORTING

2.10.1 LEARNING-AUGMENTED ALGORITHMS

Learning-augmented algorithms are classical algorithms equipped with an external prediction oracle. The oracle may be inaccurate, but the algorithm is designed to benefit when predictions are good and to degrade gracefully when they are bad. This framework has been applied to caching, scheduling, and several combinatorial problems, and it has recently been applied to sorting (Antoniadis et al., 2023).

2.10.2 SORTING WITH PREDICTIONS

Sorting with Predictions (NeurIPS 2023) studies sorting in a learning-augmented model. In one variant, each item has a predicted sorted position; in another, quick

approximate comparisons are available. The work modifies the sorting process itself to use these predictions to reduce comparison complexity (Bai & Coester, 2023).

Table 2.8: Comparison of Sorting with Predictions

Aspect	Sorting with Predictions	This Thesis
Prediction target	Element positions or comparison information	Fastest sorting algorithm
Prediction used	During sorting	Before sorting
Algorithm modified?	Yes	No
Main objective	Reduce comparison complexity	Reduce runtime by selecting algorithm
Relationship	Learning + sorting	Algorithm selection for sorting

Table 2.8 demonstrates that Sorting with Predictions modifies the internal mechanics of sorting algorithms using predictions, whereas the current work uses predictions at a higher level to choose among existing algorithms without modifying their implementation.

2.10.3 ALPHADEV

AlphaDev (Nature, 2023) formulated the search for faster sorting routines as a single-player game and trained a deep reinforcement learning agent to discover small sorting algorithms that outperformed previous human-written routines at the instruction level. AlphaDev does not select among existing algorithms; it generates new low-level sorting code.

Table 2.9: Comparison of AlphaDev

Aspect	AlphaDev	This Thesis
Learning type	Reinforcement learning	Supervised learning
Output	New low-level sorting routines	Sorting algorithm choice
Level	Instruction / code level	Array-instance level

Goal	Improve implementation	Select fastest existing algorithm
Candidate algorithms	Not a portfolio selector	Uses candidate sorting portfolio

Table 2.9 above shows the positions of AlphaDev relative to the methodology of this thesis which focuses on automatically discovering new low-level sorting code, while this thesis operates at the algorithm selection level by choosing among well-established, production-grade sorting algorithms using supervised machine learning.

2.10.4 ALGORITHM DISCOVERY VS ALGORITHM MODIFICATION VS ALGORITHM SELECTION

The three lines of work; algorithm discovery, algorithm modification and algorithm selection, can be distinguished by what they change. AlphaDev discovers new algorithms. Sorting with Predictions modifies the internal behaviour of existing algorithms using predictions. This thesis selects among existing, unmodified algorithms based on input features. The three approaches are complementary rather than competing (Mankowitz et al., 2023b).

2.11 REAL-WORLD NUMERIC ARRAYS AND DOMAIN-SPECIFIC PATTERNS

2.11.1 SYNTHETIC ARRAYS FOR CONTROLLED EVALUATION

Given that they enable the analyst to isolate the impact of specific structural properties on sorting performance while keeping all other properties constant, synthetic arrays play a crucial methodological role in per-instance sorting algorithm selection research. A baseline with no dominant values, no order, and a roughly uniform value distribution is provided by a totally random array of length n . The maximal presortedness scenario is provided by a fully sorted array. The maximum inversion count is obtained from a reverse-sorted array. Between these extremes are arrays with adjustable inversion density, piecewise-sorted arrays, and partially sorted arrays. Systematic variation along the dimensions of value repetition and distributional shape is provided by duplicate-

heavy arrays and arrays derived from particular statistical distributions (uniform, Gaussian, Zipfian, Pareto).

The literature on algorithm selection has shown the benefits of this regulated method. (Tanabe, 2022) shows that the kind and variety of synthetic instances used for training and evaluation significantly influence which algorithms appear dominant and which features appear informative in a benchmarking study of feature-based algorithm selection systems for black-box optimisation published in IEEE Transactions on Evolutionary Computation. Timsort's performance advantage on partially sorted inputs would not be shown by a training set made up only of random arrays, which would prevent the selector from learning that relationship. In a study on instance space analysis methodology, (Den Hengst et al., 2022) also observe that synthetically generated instances are crucial for attaining broad generalisation since they cover the instance space more uniformly than convenience samples from a single domain.

In this thesis, synthetic arrays are used for three distinct purposes: (1) to fill in structural areas of the instance space that might be under-represented in real-world datasets (e.g., arrays with extreme inversion ratios or very high duplicate densities that do not naturally arise in telemetry or financial data); (2) to confirm that the trained selector recovers known sorting theory relationships (e.g., that it correctly predicts Timsort for nearly sorted arrays and Quicksort for random arrays); and (3) to provide a controlled regime for sensitivity analysis.

2.11.2 REAL-WORLD NUMERIC ARRAYS

The structural complexity of arrays that emerge in deployed software systems cannot be well represented by synthetic arrays, notwithstanding their usefulness for controlled testing. Patterns found in real-world numerical data are not the result of ordinary random generators; rather, they are the result of economic, engineering, and physical processes. Continuous physical dynamics produce smooth monotone areas in sensor data, mechanical discretisation is reflected in discrete repeated values in gear position channels, and price rounding standards produce clustered value repeats in financial tick data. Because the structural characteristics of real-world arrays might be outside

the convex hull of the synthetic training examples, a selector trained solely on synthetic arrays might not be able to generalise to these real-world patterns.

This encourages the use of real-world numerical arrays as an additional source of evaluation and training data. In a study of how training instance selection affects algorithm selection models for black-box numerical optimisation, (Dietrich et al., 2024) empirically demonstrate that biased training sets consistently reduce generalisation and that training sets that combine artificially generated instances with real-world instances consistently generate more robust selectors than either alone. The hybrid synthetic and real-world dataset design used in this thesis is directly motivated by this finding.

This thesis uses real-world arrays from five different domains: seismic recordings, financial price series, meteorological data, cryptocurrency price information, and Formula 1 telemetry. Every domain produces arrays with a unique structural profile that systematically deviates from artificial standards. Each domain is introduced and its contribution to the training and assessment dataset's informative variety is explained in the next subsections.

2.11.3 FORMULA 1 TELEMETRY

One of the most architecturally varied sources of high-frequency numerical data in the public domain is Formula 1 race telemetry. Within a single race lap, each car records dozens of physical channels at sampling rates of up to several hundred hertz, generating time-indexed numerical sequences that span discrete state variables, binary indicators, near-monotonic channels, and smooth continuous signals. In recent machine learning studies of Formula 1 performance, such as the pit-stop decision-support system created by (Sasikumar et al., 2025) using Bi-LSTM models and published in *Frontiers in Artificial Intelligence*, the FastF1 Python library makes per-lap, per-channel telemetry available for analysis.

The main importance of F1 telemetry channels as a case study is their variability from the standpoint of sorting method selection. The thesis shows that the trained selector correctly routes various channel types to various algorithms inside a single real-world data source because a single lap's telemetry produces numerous arrays with essentially

distinct structural profiles. For instance, the Speed channel favours run-adaptive algorithms by capturing smooth acceleration and deceleration profiles that result in extended monotone runs with little local variance.

The nGear channel has very low cardinality, high duplicate density, and very low entropy since it maintains discrete integer gear positions. The binary DRS channel records whether the drag-reduction system is active (1) or inactive (0), resulting in an array with a high duplication density that is nearly entirely composed of two values. An array with `adj_sorted_ratio` and `longest_run_ratio` approaching 1 is produced by the Distance channel's almost monotonic growth.

F1 telemetry is a useful case study because of this within-domain variability, which tests the selector's ability to accurately route and differentiate fundamentally distinct arrays from the same physical platform. In a study using machine learning to predict Formula 1 race results using telemetry-derived features that was published in the Journal of Computer Sciences Institute, (Krzysztoń & Smółka, 2026) confirm that feature engineering from telemetry channels necessitates careful attention to each channel's unique statistical character because channels from the same lap exhibit fundamentally different distributions.

2.11.4 FINANCE, WEATHER, CRYPTOCURRENCY, AND SEISMIC DATA

The F1 telemetry case study is supplemented by four more real-world domains that offer numerical arrays with structural profiles that systematically deviate from the smooth, discrete, and monotonic patterns of vehicle telemetry. These domains ensure that the training and evaluation dataset encompasses a broad range of structural diversity. They include abrupt event-driven dynamics, extreme tail behaviour, steady seasonality, and irregular volatility.

Financial time series, such as daily or intraday samples of stock prices, indices, and exchange rates, are known for their noisy, volatile, and heavy-tailed returns. Financial price series typically have substantial excess kurtosis compared to a Gaussian distribution, moderate to high dispersion, and moderate skewness (typically negative for equity indices due to asymmetric crash risk). Due to prices' seldom precise repetition, which results in high entropy and modest cardinality, duplicate values are

rare in daily series. The daily noise of the financial markets is reflected in the moderate local fluctuation as measured by `mean_abs_diff_norm`. These characteristics typically result in arrays that are neither notably challenging for random-pivoting methods (no severe duplication concentration) nor easy for run-adaptive algorithms (no extended monotone sections).

Smooth, steady trends, significant seasonal periodicity, and comparatively low volatility are characteristics of weather time series, which include temperature, precipitation, pressure, and humidity recorded at regular intervals. In an Algorithms journal article on machine learning techniques for seasonal time series forecasting, (Singh et al., 2023a) point out that seasonal data differs from financial or random data due to its substantial autocorrelation and gradual change. With the exception of storm events, arrays formed from single-season weather channels often exhibit a near-Gaussian distributional shape, a moderate `adj_sorted_ratio` that reflects seasonal warming or cooling patterns, a moderate run structure, and a low outlier density. Because the seasonal monotone trend generates long natural runs, this profile tends to favour run-adaptive algorithms over random-pivot techniques.

The extreme end of the financial data volatility spectrum is represented by cryptocurrency price series. Heavy-tailed return distributions with power-law tails, excess kurtosis significantly higher than that of equities markets, and frequent dramatic price spikes are well-documented characteristics of Bitcoin, Ethereum, and other prominent cryptocurrencies (Tang et al., 2025a) Even with Tukey's $1.5 \cdot \text{IQR}$ fence applied, a daily Bitcoin price series over several months will usually show very high `kurtosis_excess_t`, considerable positive or negative skewness depending on the period, large dispersion, and non-negligible `outlier_ratio`. These characteristics assess the selector's capacity to manage distributional extremes that are rare in both F1 telemetry and synthetic benchmarks.

Ground motion time series from accelerometers and seismographs, or seismic recordings, are distinguished by extended periods of silence interspersed with sudden, high-amplitude occurrences. Ground motion is almost consistent during quiet times, resulting in lengthy runs of almost similar values and very little local fluctuation. A

pattern of severe outliers in relation to the background noise level is created during an earthquake occurrence when ground acceleration suddenly increases to high positive or negative values and subsequently decreases. Long near-constant runs and abrupt extreme values combine to produce arrays with high `adj_sorted_ratio` (during quiet periods), high `outlier_ratio` (during events), and high local variation (`mean_abs_diff_norm`) close to the beginning of an event. These structural features are the main obstacle for any automated processing of seismic time series, according to (Mousavi & Beroza, 2023)

2.11.5 RELATIONSHIP BETWEEN DOMAIN STRUCTURE AND FEATURE VALUES

The concept that domain-specific structural profiles generate consistently distinct feature signatures and that these feature signatures predict distinct optimal sorting methods is the theoretical foundation of the multi-domain dataset design. By tracking the expected feature values for each domain and the resulting algorithm preference suggested by the sorting literature examined in Sections 2.2 through 2.6, this subsection clarifies this notion.

Amongst all the domains in the thesis, F1 telemetry channels have the most diverse structural profiles and, as a result, the most diverse anticipated algorithm labels. The sorting literature predicts that Timsort's run-detection mechanism will virtually eliminate comparisons on such input, making it the predicted label. For the Distance channel, `adj_sorted_ratio` near 1 and `longest_run_ratio` near 1 produce a feature vector that strongly resembles a nearly-sorted array. `Duplicate_ratio` near 1, `top1_freq_ratio` near 0.9, and `entropy_ratio` near 0 for the DRS channel result in a feature vector characteristic of a near-constant array; the predicted label depends on which algorithm in the portfolio manages extreme duplicate density the best, usually an in-place algorithm that avoids Mergesort's auxiliary allocation overhead or a three-way partitioning variant.

High entropy, high kurtosis, moderate dispersion, and low `adj_sorted_ratio` together result in a feature signature for financial arrays that is most similar to a random array with heavy tails. According to the sorting literature, algorithms with deterministic

worst-case constraints, like Heapsort or Introsort, are more competitive than they would be on really uniform random input when heavy-tailed pivots weaken randomised Quicksort techniques. A feature profile typical of a minimally pre-sorted array with a near-Gaussian value distribution is produced for weather arrays by the moderate `adj_sorted_ratio` and low `outlier_ratio`. Because the monotone segments in weather time series are shorter and less prominent than in odometer-type channels, the sorting literature predicts that Timsort's run-detection will offer a moderate advantage over pure comparison-sort strategies. However, this advantage will be smaller than on F1 Distance data. A moderate improvement from run-adaptive sorting is consistent with (Singh et al., 2023b) description of seasonal time-series data as displaying strong gradual tendencies.

The extremely high `kurtosis_excess_t`, high `outlier_ratio`, and high `dispersion_ratio` for cryptocurrency arrays result in a feature signature that sets them apart from all other domains. This combination puts cryptocurrency arrays in an area of the feature space where Quicksort sensitive to pivot quality under heavy tails and Mergesort sensitive to merge imbalance near outliers face systematic difficulties. This could favour Introsort's hybrid fallback or Heapsort's data-oblivious selection strategy. This distributional extreme is not a sampling artefact, according to (Tang et al., 2025b) who document the heavy-tailed q-Gaussian character of Bitcoin returns with kurtosis significantly higher than those of stock markets.

The dual-regime structure of seismic arrays, which consists of lengthy quiet intervals with nearly constant values interspersed with sudden high-amplitude occurrences, results in a feature vector that is challenging to describe using single-value statistics. Because the calm baseline contributes long, virtually constant runs, the `outlier_ratio` will be non-negligible, the `mean_abs_diff_norm` will be high close to event beginning but low during quiet intervals, and the overall `adj_sorted_ratio` will be moderate. Real seismic recordings are used to test whether the selector can handle arrays whose structural profile is a mixture of several distinct regimes rather than a single uniform pattern because of this dual-regime character, which means that the feature vector of a seismic array does not closely resemble any single synthetic array type.

2.12 DOMAIN GENERALISATION AND ROUTED MODELS

2.12.1 MOTIVATION FOR DOMAIN GENERALISATION

The distribution of inputs received during inference may be different from the distribution of inputs used to train the model, which is a basic challenge when implementing any supervised classification model. This problem manifests itself in the context of sorting algorithm selection as follows: a model trained mainly on arrays from one data domain, such as Formula 1 telemetry, may learn selection rules that are accurate for the characteristic feature profile of that domain but consistently fail when applied to arrays from a structurally distinct domain, such as seismic waveforms or cryptocurrency price series. This behaviour is an example of distribution shift, more precisely covariate shift, where the conditional label distribution $P(Y|X)$ stays roughly constant while the marginal distribution of the input features $P(X)$ varies between the training and test settings (Guo et al., 2022)

The domain analysis in Section 2.10 makes it easy to predict the actual reality of covariate shift in sorting algorithm selection. F1 distance arrays are almost entirely sorted; weather arrays have low kurtosis and smooth structure; financial arrays have high kurtosis_excess_t and high outlier_ratio; and seismic arrays alternate between flat noise and impulsive spikes. Although split rules that roughly divide the feature space may be developed via a global XGBoost classifier trained on a balanced combination of various domains, the globally optimal boundary learned from the mixture may not match the locally optimal boundaries for each domain. The end product is a model that, on average, is acceptable but unsatisfactory on each unique domain exactly the regime in which routed or domain-specific models can outperform a single global model.

In an overview of domain generalisation techniques, (Wang et al., 2023) define domain generalisation as the challenge of learning a model from several source domains that generalises to unknown target domains without having access to target data during training. The six data domains (F1 telemetry, financial, weather, cryptocurrency, seismic, and mixed) are the training source domains, and a deployable selector must generalise to arrays whose domain identity may be unknown at inference time. This

framing directly relates to the thesis. Therefore, domain generalisation is not a side issue in this thesis but rather a fundamental prerequisite for any sorting algorithm selector that can be used in practice.

Diverse feature-space areas have varying degrees of domain generalisation deterioration. Any sufficiently trained model is likely to route arrays in structurally unambiguous regions, such as nearly entirely sorted arrays (F1 distance channel) or near-constant arrays (DRS channel), since the optimal technique is evident from the feature values alone, independent of domain. A global model trained on a mixture may learn a suboptimal average rather than the proper domain-specific rule in intermediate or ambiguous regions, where the optimal algorithm depends on fine-grained distributional properties that differ across domains.

2.12.2 WITHIN-DOMAIN AND CROSS-DOMAIN EVALUATION

This thesis evaluates the sorting algorithm selection using two different evaluation protocols: within-domain evaluation and cross-domain assessment. Together, they offer a comprehensive view of both optimistic and pessimistic constraints on deployment-time accuracy, each measuring a distinct facet of the selector's performance.

Using a typical held-out test split, within-domain evaluation trains and tests the selector on data from the same domain. In the best case scenario for any supervised model, it evaluates the selector's performance assuming that the deployment distribution is the same as the training distribution. High within-domain performance indicates that the model architecture and feature set are sufficient to capture the structural characteristics of arrays from that domain that are relevant to performance. In contrast, low within-domain performance would indicate that either the underlying performance differences are too small to be reliably exploited, or the feature set is insufficient to distinguish across methods inside that domain.

With no target-domain data utilised during training, cross-domain assessment checks the selector on a different target domain after training it on one or more source domains. It gauges how well the learnt selection mapping transcends structural changes. High cross-domain performance suggests that the selection rules acquired

from source domains transfer successfully, either because the target domain's feature-space region is covered by the source domains collectively or because the pertinent features are invariant across domains. A domain-routing architecture or data augmentation using target-domain arrays are motivated by low cross-domain performance, which shows that the target domain is located in a feature-space region where the source-trained model has learnt erroneous or absent rules.

In their study of generalisation in algorithm selection for continuous black-box optimisation, (Petelin & Cenikj, 2025) formally show that within-domain accuracy consistently overestimates cross-domain accuracy, and that the difference grows as the source and target domain distributions' structural distance increases. The use of both evaluation protocols in this thesis is directly motivated by their findings: reporting only within-domain accuracy would provide an overly optimistic picture of the selector's practical utility, whereas cross-domain evaluation shows where the selector fails and directs the design of the routed model outlined in Sections 2.11.4 and 2.11.5.

2.12.3 LEAVE-ONE-DOMAIN-OUT TESTING

A methodical evaluation procedure known as “leave-one-domain-out” (LODO) testing alternates over all of the domains that are available, each time keeping one domain out of training and utilising it as the test domain. LODO generates D distinct evaluation findings with D domains, each of which measures cross-domain generalisation to a distinct target domain. The variance among folds indicates which domains are simpler or more difficult to generalise to, while the mean performance across the D folds provides an overall evaluation of the selector’s generalisation capabilities to unexplored domains.

In algorithm selection and domain generalisation research, LODO is the common stress test for generalisation. The leave-one-domain-out strategy is the main evaluation protocol used by the NICO++ benchmark (Khorasgani et al., 2022) a canonical domain generalisation evaluation suite, which notes that it is the closest approximation to the realistic deployment scenario in which the test domain is unknown at training time. In a study on reconsidering the evaluation protocol for domain generalisation, (Liu et al., 2021) contend that LODO on multiple held-out domains concurrently, as

opposed to a single held-out domain, offers more accurate estimates of generalisation capability. However, single-domain LODO is still the norm because most benchmark suites have a small number of available domains.

In the context of sorting algorithm selection, LODO provides a practical response to the following query: how well does the selector choose sorting algorithms for arrays from the sixth domain if it is trained on five of the six accessible domains and then deployed on those arrays without any retraining? When a trained selector is applied to a new data source that was not observed during training, such as a new kind of engineering sensor data, a previously unseen financial instrument, or a new kind of scientific measurement, this is exactly the deployment scenario that arises. The best assessment of the selector’s performance in a particular target domain is given by the LODO result.

For the size of the dataset utilised in this thesis, the computational cost of LODO, which is D times the cost of a single train-test cycle, is reasonable. Using a leave-one-problem-class-out evaluation protocol that is structurally identical to LODO, Tanabe (2022) demonstrates that this approach provides more reliable estimates of generalisation capability than within-domain evaluation alone.

2.12.4 GENERAL, DOMAIN-SPECIFIC, AND ROUTED SPECIALISED MODELS

This thesis compares three different model architectures a single global model, a group of domain-specific models, and a routed specialised model as methods for choosing sorting algorithms across several domains. Different trade-offs between the need for training data, inference cost, generalisation resilience, and forecast accuracy are represented by each design.

Using the combined training data from all domains, the single global model trains a single XGBoost classifier, which is then applied consistently to all arrays at inference time, independent of domain. Because only one model is learned and stored for inference, this architecture has the least amount of training and deployment complexity. Its key drawback is that it has to learn a single feature space partition that works for a variety of fundamentally different domains. The global model can learn

the right rules for both regions when domains generate feature profiles in different, non-overlapping regions (e.g., F1 distance arrays with `adj_sorted_ratio = 1` versus bitcoin arrays with significant kurtosis). However, the global model may learn a compromise that is not ideal for each domain when domain-specific selection criteria are present in overlapping feature-space areas.

Using only the training data from that domain, domain-specific models train a single XGBoost classifier. Without dilution from data from other domains, each model is optimised for the feature-distribution and algorithm-performance landscape of its own domain. Although this architecture achieves the maximum within-domain accuracy, it necessitates knowledge of the domain identity at inference time in order to choose the appropriate domain-specific model prior to prediction. Domain-specific models cannot be used without first identifying the domain, which is the typical deployment situation. Additionally, they are costly to maintain because each model is trained on a smaller dataset than the global model, and adding a new domain necessitates training a new model.

Through employing a two-stage architecture, the routed specialised model combines the benefits of both methods: a domain classifier first determines the input array's domain identity from its feature vector, and a domain-specific algorithm selector subsequently determines the fastest sorting algorithm conditioned on the identified domain. The Mixture-of-Experts (MoE) paradigm from deep learning, in which a gating network passes each input to a subset of expert subnetworks that specialise in various input types, served as the model for this design. In a survey of MoE architectures for big language models, (Zhang et al., 2025) point out that the fundamental idea of MoE that specialised models perform better than a single generalist model when inputs are structurally heterogeneous applies widely to supervised learning domains. The routed architecture in this thesis is driven by the same realisation.

The routing phase is a classification issue in and of itself: determine which of the six domains an array most closely resembles given its 22-dimensional feature vector. Because the six domains generate distinctive feature profiles that are partially

separable in feature space this problem is well-defined. Prior to the whole routing process, the domain classifier is trained on labelled domain data and assessed for domain classification accuracy. To outperform the global model, the routed model must have high domain classification accuracy. If the domain classifier frequently makes mistakes, the wrong expert is assigned, which could result in predictions that are worse than those of the global model.

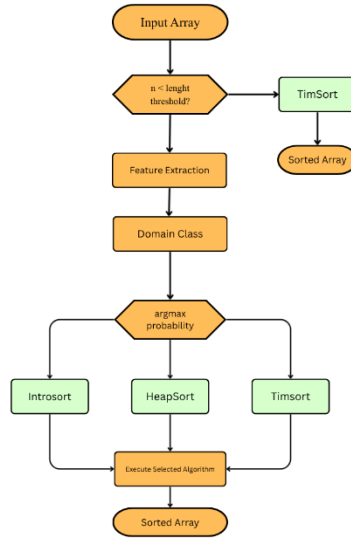


figure 2.1: Global model architecture for cross-domain sorting algorithm selection

2.12.5 ROUTED SPECIALISED MODELS FOR F1 TELEMETRY

Since Formula 1 telemetry is the most structurally heterogeneous single domain, it serves as the main case study for routed specialised models in this thesis. The seven telemetry channels speed, throttle, RPM, nGear, DRS, distance, and X/Y/Z produce feature profiles that are so dissimilar from one another that a single model trained on all F1 channels at once would encounter the same within-domain version of the cross-domain generalisation problem outlined in Section 2.11.1. Despite having nearly no structural similarities, a speed array (smooth, multi-modal, moderate entropy) and a DRS array (near-binary, entropy ≈ 0) are both accurately classified as F1 telemetry.

Therefore, a single F1-specific model needs to learn how to route between channels inside the same domain as well as between domains.

A two-level routing architecture is introduced by the routed specialised model for F1 telemetry. Based on the complete feature vector, the global domain classifier at the first level recognises the input as F1 telemetry. Using the same 22-dimensional feature vector, a channel-level router in the F1 domain classifies the array at the second level as belonging to one of the seven F1 telemetry channels. Each array in the labelled F1 data is annotated with its originating channel, which is used to train the channel router. After the channel has been identified, the fastest algorithm is predicted by a channel-aware sorting algorithm selector that has been trained on arrays from that particular channel.

The feature analysis in Section 2.10.3 provides the theoretical basis for channel-level routing in F1 telemetry. Timsort on a nearly sorted array and any method on a near-constant array will always win or tie, making distance arrays and DRS arrays practically deterministically sortable. For these channels, channel-aware selectors can use simple models to achieve almost perfect accuracy. The advantage of routing is concentrated in the ambiguous channels, such as speed, RPM, and geographical coordinates, where selection necessitates fine-grained feature discrimination and several algorithms are truly competitive. These arrays are routed to a channel-specific model that has been optimised for that specific structural regime by the channel router.

The more general Mixture-of-Experts framework can be used to comprehend this two-level routing strategy. In their study of structured specialisation in MoE systems, Cai et al. (2024) show that when the input space has a natural hierarchical structure, hierarchical routing in which a coarse router first identifies a broad category and a fine-grained router then chooses a specialised expert within that category consistently performs better than flat single-level routing. The structure of the F1 telemetry domain is precisely this: domain (F1) $\rightarrow$ channel (speed, gear, etc.) $\rightarrow$ algorithm selection choice, where each step lessens uncertainty for the subsequent one.

A lightweight XGBoost model trained on the same 22-dimensional feature vector serves as the channel classifier. The whole two-level routed pipeline's ceiling

performance is determined by its correctness. For instance, misclassifying a speed array as a gear array would send it to a low-cardinality specialist that struggles with continuous smooth arrays. Therefore, a crucial diagnostic step in the evaluation of the routed F1 model is to assess channel classification accuracy independently of algorithm selection accuracy.

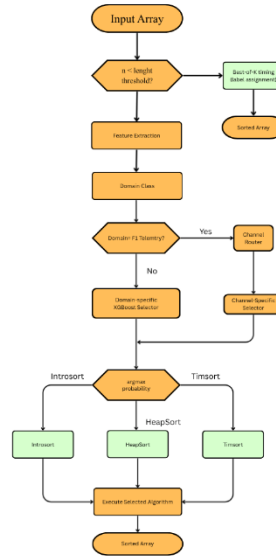


figure 2.2: Inference pipeline of the proposed sorting algorithm selector

2.13 EVALUATION METRICS FOR ALGORITHM SELECTION

2.13.1 WHY ACCURACY ALONE IS INSUFFICIENT

Classification accuracy alone is not an adequate measure for algorithm selection. A wrong prediction may be practically harmless when two algorithms have almost equal runtime, and a correct prediction may yield only a small improvement when the gap to the next algorithm is small. The runtime cost of each decision, not just whether it matched the best label must be reflected by evaluation (Kerschke et al., 2018).

2.13.2 SINGLE BEST SOLVER

The algorithm with the lowest mean (or total) runtime over the evaluation set when used uniformly across all instances is the Single Best Solver (SBS). It is the benchmark that any per-instance selector must improve upon to be useful (Tanabe, 2022).

2.13.3 VIRTUAL BEST SOLVER

The Virtual Best Solver (VBS) is the oracle that, for each instance, selects the algorithm with the lowest measured runtime. The VBS is not deployable in practice because it requires future knowledge, but it provides the upper bound against which real selectors are compared (Renau & Hart, 2024b).

2.13.4 REGRET

Per-instance regret measures the runtime cost of a wrong choice:

$$\text{Regret}_i = \text{Runtime}(\text{selected algorithm}_i) - \text{Runtime}(\text{VBS}_i)$$

Aggregate regret summarises this over the evaluation set and directly captures the runtime penalty of the selector’s mistakes.

2.13.5 GAP CLOSED

Gap closed normalises selector performance between SBS and VBS:

$$\text{Gap Closed} = (\text{SBS Runtime} - \text{Model Runtime}) / (\text{SBS Runtime} - \text{VBS Runtime})$$

A value of 0 means the selector matches the SBS, a value of 1 means it matches the VBS, and intermediate values quantify the fraction of the available improvement the selector captures (Tanabe, 2022).

2.13.6 SUMMARY OF EVALUATION METRICS

Table 2.10 below summarises the evaluation metrics used in this thesis to assess the performance of the machine learning-based algorithm selector.

Table 2.10: Summary of Evaluation Metrics Used for Algorithm Selection Assessment

Metric	What It Measures	Role in This Thesis
Accuracy	Fraction of correct algorithm predictions	Necessary but not sufficient
SBS	Best fixed algorithm baseline	Lower-cost baseline
VBS	Oracle per-instance best	Upper bound

Regret	Runtime cost of wrong choices	Runtime-sensitive primary metric
Gap closed	Fraction of SBS–VBS gap captured	Normalised selector quality

CHAPTER THREE

METHODOLOGY

3.1 RESEARCH METHODOLOGY OVERVIEW

Sorting algorithm selection is formulated in this thesis as a supervised classification problem. Given a one-dimensional numeric array, a fixed set of structural features is extracted and a classifier predicts which sorting algorithm will execute fastest on that specific input. The formulation instantiates the Algorithm Selection Problem first formalised by Rice (1976), which defines a problem space P of instances, a feature space F of measurable characteristics, an algorithm space A of candidate solvers, and a performance space Y of measured outcomes. Let $x \in \mathbb{R}^{16}$ denote the feature vector of an input array, $A = \{a_1, a_2, \dots, a_k\}$ the set of candidate sorting algorithms, and $T(a, x)$ the empirical execution time of algorithm a on the array characterised by x . The objective is to learn a selector $S : F \rightarrow A$ that satisfies:

$$a^* = \arg \min T(a, x), \quad a \in A \quad (3.1)$$

Classification was preferred to regression after early experimental iterations. A multi-output regressor that predicts absolute execution times tends to learn the dominant effect of array size and obscures the smaller relative differences that drive selection. Reformulating the task as direct prediction of the fastest-algorithm identity removes this confound and yields substantially higher accuracy. XGBoost (Chen and Guestrin, 2016) is adopted as the classifier because gradient-boosted decision trees model the non-linear interactions between size and structure that govern sorting performance, and provide built-in L1/L2 regularisation that is robust to the label noise inherent in timing measurements. XGBoost learn to converge better non linearly

3.2 SYSTEM ARCHITECTURE

The runtime system processes each array in three stages, illustrated in Figure 3.1. Sixteen structural features are computed in $O(n)$ time via a feature extraction module that first processes the raw array. The trained XGBoost classifier receives the

generated feature vector and produces probability estimates over the potential algorithms. The array is sorted using the algorithm that has the highest anticipated likelihood, which is ultimately returned as the recommendation.

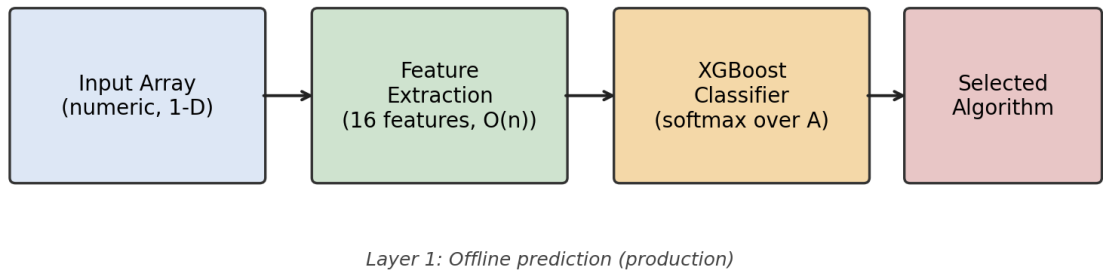


Figure 3.1: Inference pipeline of the proposed sorting algorithm selector.

In comparison to the sorting process, which runs in $O(n \log n)$, the overhead of feature extraction and prediction must continue to be minimal. The selector works best for medium-to-large arrays since the sorting cost increases with n and the prediction cost is fixed; on very short arrays, the selection overhead may outweigh any potential runtime savings. As a result, the design is positioned with a length threshold below which a default algorithm is used immediately.

3.3 ALGORITHM PORTFOLIO

Each of the three comparison-based algorithms in the portfolio is accessible at the C level using NumPy and the Python standard library, ensuring that observed differences are not the result of implementation complexity but rather algorithmic features. Introsort (Musser, 1997) is a hybrid algorithm that combines the cache-friendly access pattern of quicksort with an $O(n \log n)$ worst-case guarantee. It starts out as quicksort, switches to heapsort when recursion depth surpasses $2[\log n]$, and employs insertion sort on small sub-arrays. Heapsort (Williams, 1964) offers $O(n \log n)$ time and $O(1)$ auxiliary space in all scenarios, but it maintains non-sequential

memory accesses that are detrimental to contemporary caches. It is the conservative baseline and is rarely the quickest but never catastrophic. Timsort (Peters, 2002) is an adaptive merge sort that detects natural runs in the input and merges them using an exponential search during merge operations, achieving near-linear time on sorted or nearly sorted inputs. The three algorithms exploit fundamentally different input properties, so no single member of the portfolio dominates across realistic workloads and a feature-driven selector is potentially useful.

3.4 FEATURE ENGINEERING

Feature engineering transforms the raw array into a fixed-length vector that captures the structural properties relevant to sorting performance. The design follows two non-negotiable constraints. First, every feature must be computable in $O(n)$ time, otherwise feature extraction would be as expensive as sorting itself and the selector would carry no net benefit. Second, each feature must be informative, meaning that it captures a property linked in the literature to differential algorithmic behaviour. Sixteen features are computed and organised into five groups as summarised in Table 3.1.

Table 3.1: Structural features used for array characterisation.

Group	Feature	Definition	Targeted behaviour
Size	length_norm	$\log_{10}(n)/\log_{10}(n_max)$	Size-dependent constants and cache effects
Ordering	adj_sorted_ratio	Fraction of adjacent pairs in non-decreasing order	Timsort advantage on sorted prefixes
Ordering	inversion_ratio	Normalised inversion count (subsamped for $n > 10^4$)	Global disorder; introsort vs timsort
Ordering	runs_ratio	Number of monotonic runs / n	Run-adaptive merging in timsort
Ordering	longest_run_ratio	Length of the longest run / n	Dominant sorted region
Ordering	mean_abs_diff_norm	Mean $ x_{i+1} - x_i $ normalised by range	Local volatility
Repetition	duplicate_ratio	$1 - (\text{unique values} / n)$	Partition imbalance in quicksort
Repetition	top1_freq_ratio	Frequency of the most common value / n	Single-value dominance
Repetition	top5_freq_ratio	Combined frequency of the five most common values / n	Value clustering
Distribution	entropy_ratio	Shannon entropy normalised by $\log_2(n)$	Value diversity

Distribution	dispersion_ratio	Std. deviation / mean (coefficient of variation)	Spread relative to center
Distribution	skewness_t	$\text{sign}(s) \cdot \log(1+ s)$, s = sample skewness	Asymmetry, tail direction
Distribution	kurtosis_excess_t	$\text{sign}(k) \cdot \log(1+ k)$, k = excess kurtosis	normal or heavy tailed distribution, Positive means; more tails (differentiate rates)
Distribution	outlier_ratio	Fraction outside the $1.5 \cdot \text{IQR}$ fences	Pivot-quality risk for quicksort
Robust scale	iqr_norm	$\text{IQR} / (\max - \min)$	Robust spread
Robust scale	mad_norm	Median absolute deviation / range	Robust variability

3.4.1 NUMERICAL STABILITY AND NORMALISATION

Most features are bounded in $[0, 1]$ by construction, either as fractions of n or as ratios over the value range, ensuring that they are comparable across arrays of different size without external scaling. The exceptions are the skewness and kurtosis-excess descriptors, which are transformed using the signed logarithm

$$f(x) = \text{sign}(x) \cdot \log(1 + |x|) \quad (3.2)$$

to compress extreme values while preserving sign information. All divisions include an additive constant of 10^{-12} in the denominator to guarantee numerical stability on constant arrays, and explicit branches handle the degenerate cases of single-element and uniform-value arrays. A single canonical implementation of the feature extractor is shared across data generation, training and inference, eliminating any possibility of feature drift between offline and deployed code.

3.5 DATA COLLECTION AND LABELLING

The training corpus contains 1,188,265 numeric arrays extracted from five real-world domains chosen to maximise structural diversity: Formula 1 telemetry, equity market prices, cryptocurrency OHLCV prices, Earthquake and weather event measurements observations. Real data is used in preference to synthetic distributions because preliminary experiments showed that classifiers trained on uniform, normal and lognormal samples fail to transfer to genuine data: the correlations, trends and

domain-specific structure of empirical arrays are not reproduced by parametric generators. Figure 3.2 summarises the data pipeline.

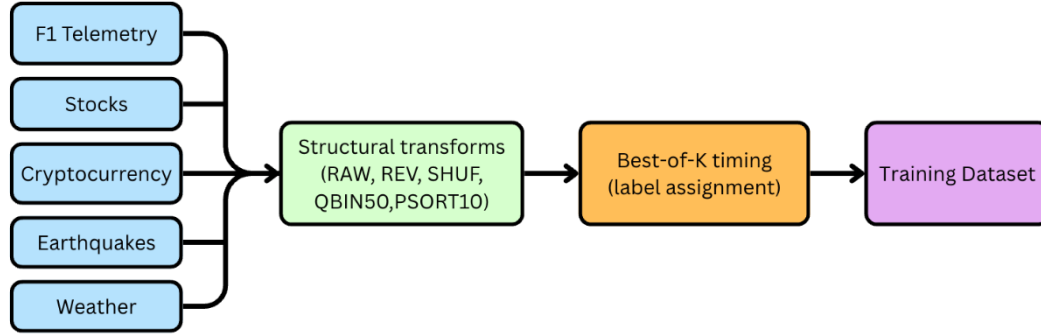


Figure 3.2: Data collection and labelling pipeline.

3.5.1 STRUCTURAL TRANSFORMS

Without modification, most real-world arrays are at least partially ordered, which would teach the classifier to always predict timsort regardless of features. To force a non-trivial decision boundary, each raw array is replicated under five structural transforms: RAW¹ preserves the original ordering; REV² fully reverses the array; SHUF³ applies a uniform random permutation; QBIN50⁴ quantises the values into fifty equal-frequency bins to inflate duplicate density; and PSORT10⁵ swaps ten percent of the elements into their correct positions to produce a partially sorted variant. The transforms induce the full spectrum of orderings on which the candidate algorithms differ most sharply.

3.5.2 LABELLING PROTOCOL

Ground-truth labels are derived empirically. Each candidate algorithm is timed on each array using a best-of-K protocol, where $K = 5$ for arrays shorter than 10^4 elements and $K = 3$ for larger arrays. Garbage collection is disabled during measurement to remove non-deterministic pauses, and the timing process is pinned to a single CPU core to avoid cache contention from context switching. The algorithm with the smallest

¹ Original data, no changes applied

² Array reversed from end to start

³ randomly shuffled, all order destroyed

⁴ Values bucketed into 50 bins

⁵ Sorted then 10% elements swapped

minimum-of-K execution time is assigned as the label for that array. When the gap between the best and second-best algorithm is below five percent the label is considered ambiguous and the row is later filtered out, except for arrays of at least 2,000 elements where timing measurements are stable enough to trust narrow margins.

3.5.3 QUALITY CONTROLS

Eight quality controls are applied before training. Duplicate arrays, rows containing NaN or Inf, and constant arrays are removed; arrays shorter than ten elements are discarded as dominated by overhead; the domain composition of the dataset is monitored to prevent any single source from dominating the training signal; a source identifier is attached to every array so that derivatives of the same time series are kept on the same side of the train–test split (preventing source-level leakage); and the validated, single source-of-truth feature implementation is reused at every stage of the pipeline.

3.6 XGBOOST CLASSIFIER

The production classifier is the final iteration of a sequence of design experiments that progressively eliminated regression-style objectives, synthetic training data, leakage-prone evaluation splits and overly aggressive regret-weighted losses. The retained architecture is a three-class (Introsort, heapsort, and timsort), and a softmax XGBoost model trained on the 16 structural features described in Section 3.4. The remainder of this section presents its training protocol, class-imbalance strategy and learnt structure.

3.6.1 CLASS IMBALANCE HANDLING

The raw label distribution is severely skewed: timsort wins on 85.0% of arrays, heapsort on 10.5% and introsort on 4.4%. Training directly on this distribution would yield a trivial ‘always timsort’ classifier. Three complementary mechanisms are applied. First, a noise filter removes ambiguous near-tie rows where the gap between the top two algorithms is below 5% (except on arrays $\geq 2,000$ elements), reducing the corpus from 1,188,265 to 1,082,547 rows. Second, random undersampling caps the majority class at three times the minority count, producing a balanced subset of

approximately 196,000 rows. Third, inverse-frequency sample weights are applied during training so that each class contributes equally to the loss.

3.6.2 TRAINING PROTOCOL

The undersampled dataset is partitioned by stratified sampling into training (70%), validation (15%) and test (15%) folds, with the validation set used for early stopping and the test set reserved exclusively for final evaluation. To prevent source-level leakage, a GroupShuffleSplit variant is applied using the source identifier of each array, so that arrays derived from the same time series are never split across training and testing datasets. The hyperparameters of the production model are summarised in Table 3.2.

Table 3.2: XGBoost hyperparameters of the production model.

Hyperparameter	Value	Rationale
objective	multi:softprob	Three-class softmax with probability outputs
tree_method	hist	Histogram splitting for training speed
n_estimators	800	Capacity with early stopping at 50 rounds
learning_rate	0.05	Conservative shrinkage for noisy labels
max_depth	8	Capture size×structure interactions
min_child_weight	5	Avoid splits on small leaves
subsample	0.85	Row sub-sampling for variance reduction
colsample_bytree	0.85	Feature sub-sampling per tree
reg_alpha	0.1	L1 regularisation
reg_lambda	1.0	L2 regularisation
random_state	42	Reproducibility

3.6.3 LEARNT STRUCTURE

Figure 3.3 below ranks the features by total gain. The length normalisation feature contributes 26.2% of total gain, confirming the dominant effect of array size on the relative performance of the candidates. Repetition features (top-5 and top-1 frequency, duplicate ratio) account for a further 28%, and ordering features (longest run, runs ratio, adjacent sorted ratio, inversion ratio) for approximately 21%. Together these three families supply the bulk of the discriminative signal, in line with the literature on input-sensitive sorting.

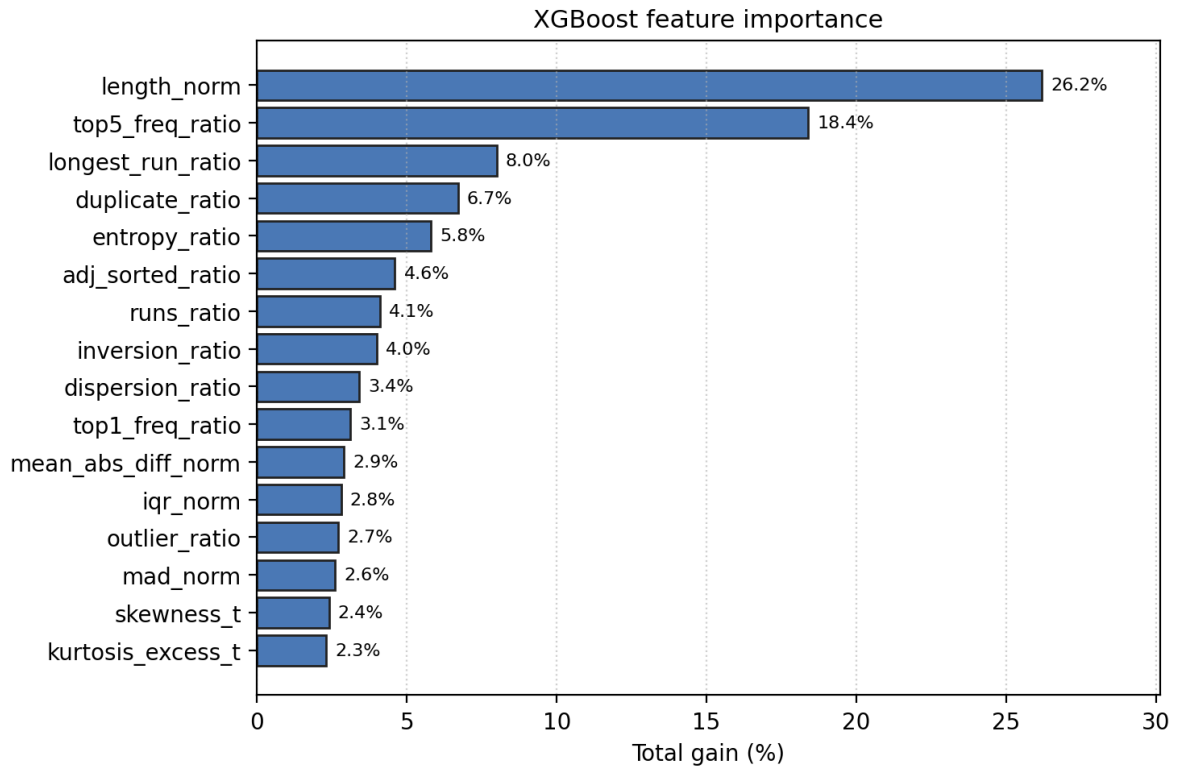


Figure 3.3: Feature importance of the production XGBoost model, measured by total gain.

The structure of the model’s errors is shown in the test-set confusion matrix of Figure 3.4. Timsort is identified with 94.5% recall; the bulk of the residual error is concentrated in the introsort–heapsort cell, where the two algorithms are nearly indistinguishable from the available structural features. This confusion is consequential only when the two algorithms differ substantially in runtime, which is rare; the practical implication is therefore that misclassifications within the introsort–heapsort pair are low-regret, whereas the more costly timsort errors are kept small by the high timsort recall.

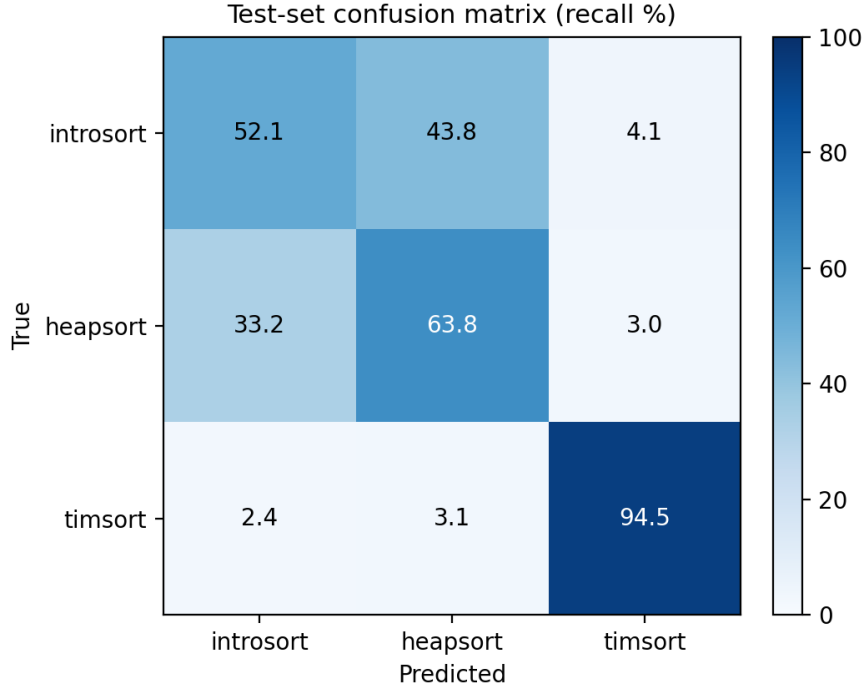


Figure 3.4: Confusion matrix of the production model on the held-out test set, rows normalised to recall.

3.7 EVALUATION PROTOCOL

Standard classification metrics are insufficient for algorithm selection because they treat all misclassifications equally, whereas the practical cost of a wrong choice depends on the runtime gap to the optimal algorithm. The evaluation protocol therefore combines classification metrics with regret-based metrics drawn from the algorithm selection literature.

3.7.1 CLASSIFICATION METRICS

Top-1 accuracy measures the fraction of arrays for which the predicted algorithm matches the empirically fastest. Balanced accuracy averages per-class recall to compensate for the residual class imbalance. Per-class precision, recall and F1 scores are reported, together with the full confusion matrix, to expose structural error patterns that are invisible to aggregate accuracy.

3.7.2 REGRET METRICS

Two reference points define the achievable performance window. The Virtual Best Solver (VBS) is the oracle that, for each array, selects the truly fastest algorithm; it

provides the unattainable lower bound on cumulative runtime. The Single Best Solver (SBS) is the one algorithm with the lowest cumulative runtime when applied uniformly to every array; it provides the naive upper bound that any selector must beat. Per-instance regret and gap closed are defined as:

$$\text{Regret} = \frac{(T_{\text{model}} - T_{\text{VBS}})}{T_{\text{VBS}}} \quad (3.3)$$

$$\text{Gap Closed} = \frac{(T_{\text{SBS}} - T_{\text{model}})}{(T_{\text{SBS}} - T_{\text{VBS}})} \quad (3.4)$$

A gap-closed value of zero indicates parity with the SBS baseline and a value of one indicates parity with the oracle. The zero-regret percentage, defined as the fraction of arrays for which the selector's choice is exactly as fast as the oracle's, complements these metrics by characterising the bulk of the error distribution. Figure 3.5 illustrates the distribution of per-instance regret obtained by the production model, in which the vast majority of predictions incur no time penalty and the residual regrets are concentrated at sub-microsecond magnitudes.

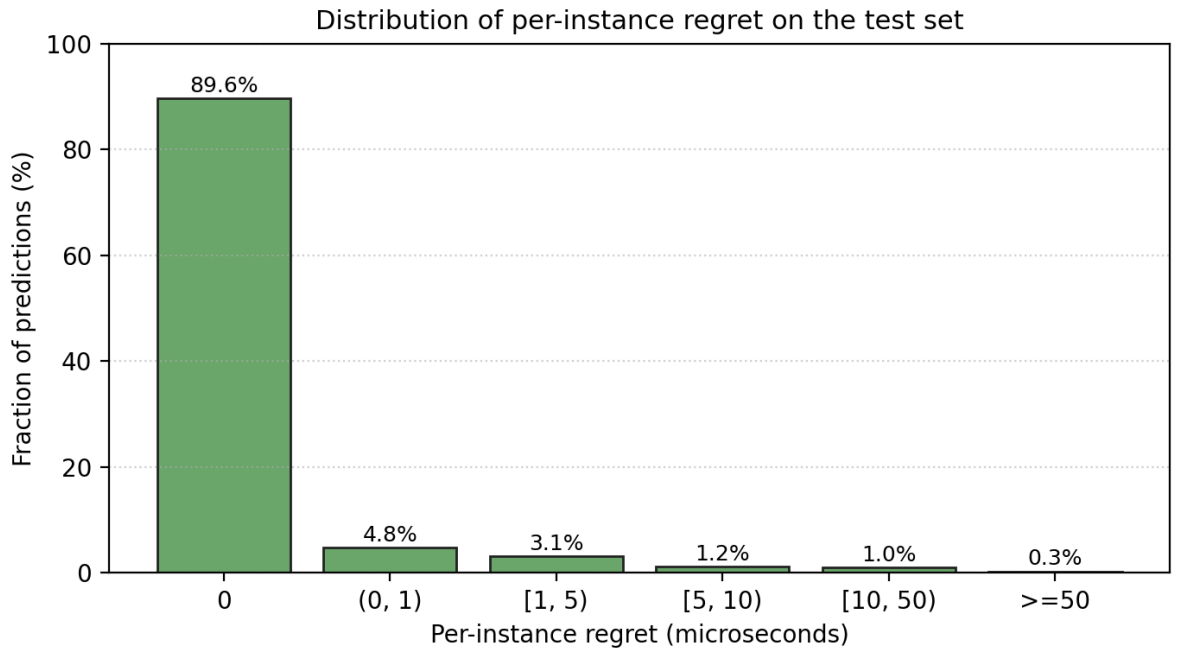


Figure 3.5: Empirical distribution of per-instance regret on the test set.

3.7.3 BASELINE COMPARISONS

Two baselines contextualise the selector. The SBS baseline always uses the empirically best single algorithm and, by construction, achieves 0% gap closed. Uniform random selection from the portfolio provides an information-free reference. Reporting the selector’s gap closed against both baselines isolates the contribution of the feature-driven decision from the contribution of the algorithm portfolio itself.

3.7.4 DOMAIN HOLDOUT EVALUATION

To assess generalisation beyond domain-specific patterns, leave-one-domain-out cross-validation is performed: in each fold the classifier is trained on four of the five domains and evaluated on the held-out fifth. Table 3.3 reports the results.

Table 3.3: Leave-one-domain-out cross-validation results.

Held-out domain	Top-1 accuracy (%)	Gap closed (%)
Formula 1	67.4	75.5
Stocks	73.1	91.2
Cryptocurrency	70.8	89.6
Earthquakes	72.5	92.4
Weather	60.6	89.7

Gap closed exceeds 75% in every fold and 89% in four of five, indicating that the selector relies primarily on transferable structural properties rather than memorised domain idioms. The weakest fold occurs when Formula 1 telemetry is held out, as expected given that it is the largest and most structurally distinctive source.

3.8 REPRODUCIBILITY

All experiments use a fixed random seed of 42 for data splitting, under sampling and model training. The feature extraction function is deterministic and shared across data generation, training and inference. Trained models are serialised in both the native XGBoost JSON format and the portable ONNX format, and the training corpus is archived as a CSV file together with its metadata schema, enabling exact replication of all reported results.

The supporting code snippets, implementation examples, and supplementary project materials are also available online at <https://fsa-phi.vercel.app/>,

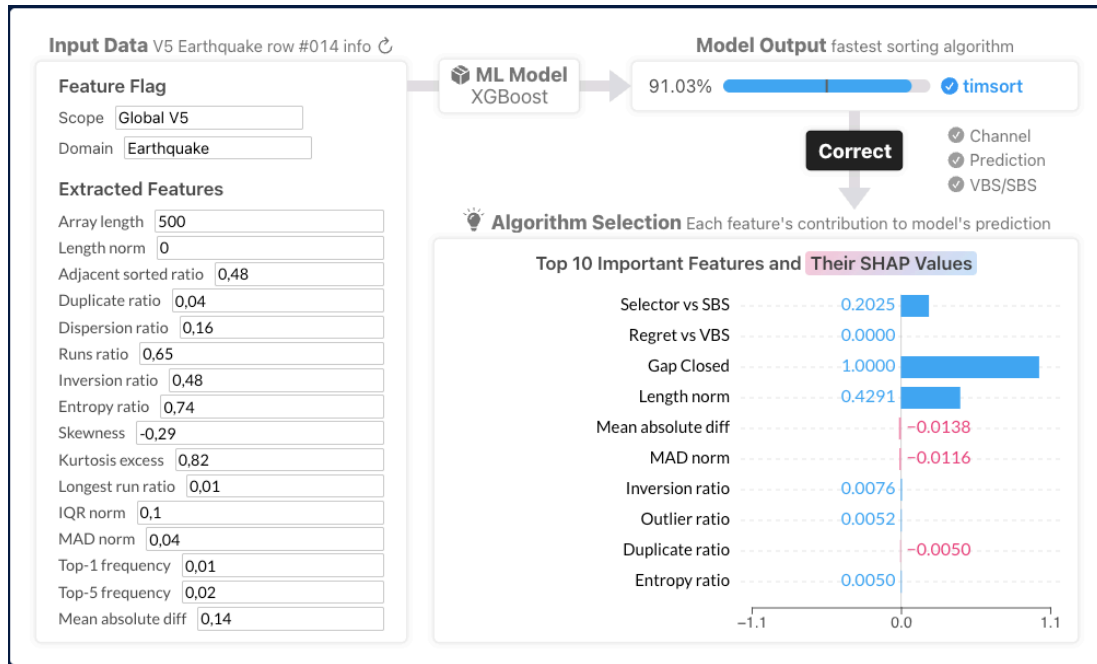


Figure 3.6: Online experiment implementation

providing an accessible reference for reproducing and understanding the experimental workflow.

3.9 CONCLUSION

This chapter presented a condensed methodology for per-instance sorting algorithm selection. The system extracts sixteen $O(n)$ structural features from a numeric array and uses an XGBoost classifier trained on 1.18 million real-world arrays from five domains to predict which of three C-level sorting algorithms (introsort, heapsort, timsort) will execute fastest. Noise filtering, undersampling, and inverse-frequency weighting are used to address class imbalance; regret-based metrics that quantify the practical cost of each action are combined with classification metrics for evaluation. The experimental results obtained under this methodology are presented and analyzed in the following chapter.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 OVERVIEW

This chapter translates the experimental results obtained from the selector developed in Chapter Three. The discussion focuses on what the observed accuracy, the discussion focuses on what the observed accuracy, regret, feature-importance, domain-holdout, and routing results make known about the viability and limits of structural-feature-based sorting algorithm selection instead of restating the full methodology.

The central issue is knowing if the selector makes decisions that reduce total runtime not simply whether the predicted label always matches the empirically fastest algorithm. That is the main reason why the chapter treats classification accuracy as a regret-based performance and diagnostic measure as the main indicator of practical value.

4.2 PRODUCTION MODEL PERFORMANCE

Using 29,494 held-out test arrays, the production classifier's top-1 accuracy was 76.1%. This precision alone points to a practical but flawed selection. The confusion pattern provides a more accurate explanation: 43.8% of introsort-labeled arrays are projected as heapsort, while timsort is detected with 94.5% recall while introsort recall is just 52.1%.

This imbalance has significance. Timsort's successful cases leave a significant mark in the ability vector because it benefits from observable input structure, particularly pre-existing order and extended runs. Due to the situations in which introsort reverts to heapsort, the behavioral difference between the two algorithms is lessened, making it more difficult to distinguish between the two algorithms. As a result, the model is weakest when the differentiation relies on lower-level execution effects and greatest when the algorithmic mechanism leaves a distinct structural trace.

The interpretation of the 76.1% accuracy figure is altered by the regret analysis. The model reduces the runtime difference between the SBS baseline of 21.267 seconds and the VBS oracle of 17.195 seconds by 93.1% throughout 1,188,265 analyzed arrays, and 89.6% of predictions result in zero regret. Put differently, a large number of label errors are not expensive. The selector is good at avoiding choices that significantly increase runtime, but it is not perfect at replicating the oracle labels.

This judgment is reinforced by the learned feature-importance structure. 75.2% of the overall benefit is attributed to array length, repetition, and ordering aspects. These feature families are precisely what are anticipated to affect sorting behavior: ordering regulates timsort's adaptive advantage, size modifies constant-factor and cache effects, and repetition modifies partition and movement behavior. Therefore, rather than being influenced by coincidental connections, the model's judgments seem to be motivated by reasonable sorting methods.

4.3 INTERPRETATION OF MODEL ITERATIONS

The model iterations demonstrate how the problem was progressively reduced to its most practical form. Given that array size dominated absolute runtime and the tiny differences between competing algorithms were more difficult to learn, the first regressor only obtained 44.4% accuracy. Accuracy increased to 62.5% when the job was reformulated as direct classification, demonstrating that predicting the winning method was more reliable than forecasting all runtimes and then comparing them.

100% accuracy is achieved by the timing-feature iteration, but thus outcome is more of a threat than a success. It demonstrates that if the model is permitted to use information that effectively exposes the solution, near-perfect discrimination exists; however, this information would negate the goal of pre-sorting selection. Thus, this iteration distinguishes between deployable usefulness and predictive possibility.

Since they demonstrate what might go wrong in evaluation, the later iterations are the most useful. Recurrent or related time series can inflate findings when split incorrectly, as seen by the source-aware split, which decreased performance to 71.2%

accuracy and 45.6% gap closed. Additionally, the regret-weighted loss failed the cleaner cross-entropy target, indicating that there was more instability than gain when the loss function was forced to pursue cost sensitivity. Therefore, the final model is not just the most accurate iteration, but also the iteration that strikes the optimum balance between runtime value, deployability, and leakage control.

4.4 THE BINARY NATURE OF THE SELECTION PROBLEM

The binary cascade experiment differentiates the main boundary in the current feature space. Stage 1, which separates timsort from the non-timsort algorithms, achieved an AUC of 0.982. Stage 2, which separates introsort from heapsort, achieved only 0.603, barely above random chance. This outcome is important because it describes the production model's confusion matrix: the selector can dependably notice when the adaptive behaviour of timsort is essential, but it has limited evidence for separating the two non-adaptive alternatives.

The weak introsort-heapsort limit should not be understood as a simple training failure. It is better understood as a feature-space ceiling. The current features describe the values and their order, but they do not directly observe cache-line use, branch prediction behaviour, memory stride patterns, or other microarchitectural properties that can decide close differences between introsort and heapsort. The available information is durable enough to identify timsort cases, but not always strong enough to resolve the non-timsort pair.

Regret-based evaluation is important because of this ceiling. If the unresolved boundary were linked with large runtime penalties, the selector would not be practically useful. Instead, most residual errors occur in a low-cost region where introsort and heapsort have similar runtimes. Strong runtime performance is delivered by the production model despite imperfect three-class accuracy.

4.5 GENERALISATION ACROSS DOMAINS

The leave-one-domain-out results are used to test whether the selector transfers beyond the domains it has already seen. This is important because a practical selector

is unlikely to be deployed only on arrays drawn from the exact same sources used during training.

Table 4.1: Leave-one-domain-out cross-validation results.

Held-out domain	Test size	Accuracy (%)	Gap closed (%)	Zero-regret (%)
Cryptocurrency	100,000	87.5	90.1	87.9
Earthquake	100,003	82.1	95.0	82.8
F1 Telemetry	885,042	88.8	75.5	89.8
Stock	100,000	88.0	90.7	88.3
Weather	3,220	60.6	89.7	61.0

Gap closed surpasses 75% in every fold and 89% in four of the five folds. This proposes that the selector has learned structural rules that move across domains instead of rules tied to only one dataset. The weakest fold is F1 telemetry, with 75.5% gap closed, which is practical because F1 contains several sharply different channel types. Holding it out removes not just one domain, but a large set of distinct structural regimes.

The Weather fold is the clearest example of why label accuracy can understate practical performance. Accuracy falls to 60.6%, yet gap closed remains 89.7%. The selector therefore makes many label-level mistakes on weather arrays, but those mistakes usually occur where the selected and optimal algorithms have similar runtimes.

The cross-domain pattern also indicates that the real-world corpus is doing useful work. The selector is tested across gradual trends, duplicate-heavy signals, heavy-tailed financial series, seismic bursts, and meteorological smoothness. Strong gap-closed values across these regimes provide better evidence of robustness than would be obtained from synthetic distributions alone.

4.6 ROUTED MODELS AND SOURCE LEAKAGE

The supplementary routed-model experiment tested whether domain or channel metadata can improve selection by routing each F1 telemetry array to a more specialised model. In non-strict evaluation, the routed model achieved 86.0% accuracy, apparently improving on the global selector. However, strict source-separated

evaluation reduced routed accuracy to 48.9%. This large drop shows that the apparent gain was driven by leakage from overlapping time series rather than by a robust metadata signal.

The main implication is that routing is especially vulnerable to memorising source identity. When related time series are split across training and testing, a routed model can appear to learn useful specialisation while actually exploiting repeated source structure. The strict result is therefore the more credible deployment estimate. Routing remains a plausible architecture, but the evidence here does not support claiming an improvement from it.

The routed-model result also clarifies the interchange between global and specialised selectors. A single global model has lower architectural complexity and appears more robust under source separation. Specialised models may still be valuable when domain identity is reliable and adequate independent training data is available for each route, but the evidence here shows metadata alone is not enough. Any future routed system must prove that its router captures stable structural differences rather than memorising repeated sources.

4.7 IMPLICATIONS OF THE FINDINGS

The results support the main thesis claim that machine-learning-based algorithm selection can improve sorting performance on realistic numeric arrays. The strongest evidence is not the 76.1% top-1 accuracy in isolation, but the combination of high timsort recall, 93.1% gap closed, and 89.6% zero-regret predictions. Together, these metrics show that the selector captures most of the practically available improvement over the best fixed algorithm while avoiding expensive errors in the regions where algorithm choice matters most.

The findings also enhance what can realistically be expected from a lightweight selector. The current methodology is highly effective at recognizing large structural regimes, especially the difference between arrays that favour timsort's run-adaptive behaviour and those that do not. It is less effective at detecting low-level performance differences between algorithms whose runtime depends on hardware-sensitive effects.

This distinction is important for deployment: the selector should be seen as a structural decision layer, not as a complete replacement for low-level performance modelling.

The broader implication is that the story is changed by evaluation design. Without strict splits, the routed experiment would appear successful whereas it reveals substantial leakage with strict splits. Without regret metrics, the production model would appear only moderately accurate; with regret metrics, it captures nearly all of the useful runtime improvement. These two observations indicate that selector quality cannot be judged from headline accuracy alone.

4.8 FUTURE DIRECTIONS

The binary ceiling recognized by the cascade experiment proposes that future research should aim at the introsort-heapsort boundary. Discriminative information that the current value-level features cannot capture may be provided by lightweight microarchitectural proxies, such as estimated cache-line crossings, memory-stride regularity, branches indicators, or sampled access-pattern descriptors .

A second direction is experimental validation of the LinUCB contextual bandit layer described in the methodology. The offline XGBoost classifier provides a strong preceding, but real arrangements may experience workload drift. Adaptation speed, cumulative regret, stability under noisy runtime feedback, and behaviour under abrupt domain shifts should be measured by a controlled bandit evaluation.

Cross-platform evaluation is a third direction. The current findings are tied to the hardware and software environment used for timing. Testing the same pipeline on ARM processors, different cache hierarchies, and alternative Python or NumPy builds would show whether the learnt feature importances are hardware-invariant or whether platform-specific retraining is required. This is especially important because the unresolved introsort-heapsort boundary is likely to depend on microarchitectural behaviour.

4.9 CHAPTER SUMMARY

This chapter has shown that the selector is practically useful despite imperfect top-1 accuracy. It closes 93.1% of the SBS-to-VBS runtime gap and produces zero regret on 89.6% of predictions. The main residual weakness is the introsort-heapsort boundary, which appears to require information beyond the current value-level feature set. The domain-holdout results support cross-domain robustness, while the routed-model experiment shows the risk of leakage when evaluation is not source-separated. Overall, the results support the thesis that cheap structural features can deliver meaningful per-instance sorting improvements when judged with runtime-sensitive metrics.

CHAPTER FIVE

CONCLUSION

This thesis addressed per-instance algorithm selection for sorting on real-world numeric arrays. The selector extracts sixteen $O(n)$ structural features from each array and uses an XGBoost classifier to choose, before sorting begins, between three C-level comparison-based algorithms exposed by NumPy and the Python standard library: introsort, heapsort, and timsort. The classifier was trained and evaluated on a corpus of 1,188,265 arrays collected from five structurally diverse domains: Formula 1 telemetry, equity prices, cryptocurrency prices, seismic measurements, and weather observations; under a strict, source-aware evaluation protocol.

The production model attains 76.1% top-1 accuracy on the held-out test set and closes 93.1% of the runtime gap between the Single Best Solver and the Virtual Best Solver. The zero-regret rate is 89.6%, which means that nearly nine out of ten predictions experience no execution-time penalty relative to the oracle, with a mean per-instance regret of approximately 0.23 microseconds. A secondary structural outcome appeared from the model-evolution experiments: timsort is highly separable from the other two candidates on the basis of structural features alone, whereas there is nearly no distinguishable difference between introsort and heapsort (binary AUC ≈ 0.603). Significantly, the residual misunderstanding is concentrated in this low-regret cell, so the deficient three-class classifier still delivers near-oracle end-to-end runtime.

The thesis makes three clear contributions, aligned with the objectives stated in Section 1.4:

- **A deployable selector.** The majority of the attainable runtime increase above any single fixed solution on real-world workloads is captured by a compact pipeline that combines inexpensive structural characteristics with a gradient-boosted classifier.
- **A rigorous evaluation framework.** When combined, source-level train-test separation, near-tie noise filtering, leave-one-domain-out cross-validation, and runtime-sensitive metrics (per-instance regret, gap closed, zero-regret rate)

provide a more accurate representation of selector quality than classification accuracy alone.

- **A reproducible artefact.** One source-of-truth feature extractor, a deciding data pipeline, and models generated in both XGBoost JSON and ONNX formats are all included in this Python implementation, which is packaged so that each reported result can be replicated from the archived corpus.

Since sorting is done continuously and at scale across machine-learning pre-processing pipelines, analytics platforms, and database engines, even tiny per-instance gains add up to quantifiable reductions in latency, quantity, and infrastructure costs. The selector demonstrates that these gains can be obtained without modifying the underlying sorting routines, specialised hardware and imposing per-call overheads that would outweigh the benefit on production-sized arrays. Beyond sorting, the cost-aware evaluation methodology, particularly the joint use of regret-based metrics and source-aware splits, is directly transferable to other instances of the Algorithm Selection Problem where runtime rather than label identity, is the quantity of interest.

Several limitations of the current study should be recognised. The algorithm portfolio is limited to three comparison-based sorts accessible at the C level through NumPy and the Python standard library; non-comparison sorts and custom low-level implementations are deliberately excluded, in line with the scope defined in Section 1.3. All timing measurements were collected on a single hardware configuration with garbage collection disabled and CPU affinity pinned, so absolute regret figures should be interpreted as ceiling estimates rather than as universal guarantees across architectures. The online adaptation layer (LinUCB contextual bandit) was designed and integrated but not empirically validated, and the source-aware evaluation protocol revealed a residual sensitivity to domain composition: accuracy drops by roughly five percentage points relative to the unrestricted split, indicating that some of the apparent generalisation strength rests on dataset-specific structure. Finally, feature-extraction overhead becomes unprofitable for arrays shorter than approximately one thousand elements, below which a fixed default algorithm remains the rational choice.

5.1 FUTURE WORK

The limitations above, together with several questions brought up but not resolved within the scope of this thesis, provides three concrete directions for future research.

The selector should be reassessed under a considerably larger portfolio. Promising additions include Powersort and its multiway variants, which generalise timsort’s run-adaptive merging; non-comparison sorts such as radix and bucket sort, guarded by domain checks that confirm the required key-structure assumptions; SIMD-aware and branch-free sorting kernels that exploit modern instruction sets; and GPU-resident sorts for very large arrays. Enlarging the portfolio is expected to expose new winners on workloads currently dominated by timsort, but it also raises interesting open questions about how portfolio size affects classifier calibration, training-data requirements, and the amortised cost of decision-making at inference time.

The LinUCB contextual bandit designed in Chapter Three was presented as a deliberate future direction rather than a validated contribution. A complete experimental evaluation is needed, covering convergence speed, cumulative regret, and adaptation behaviour under controlled distribution-shift scenarios such as gradual workload drift, abrupt domain change, and adversarial input mixes. Related questions include the value of warm-starting the bandit from the XGBoost posterior probabilities, the relative merits of alternative exploration strategies (for example Thompson sampling or neural contextual bandits), and the practical cost of collecting reliable runtime feedback in production pipelines where measurement noise and asynchronous execution complicate reward attribution.

The leave-one-domain-out experiments showed that the selector transfers reasonably well across domains but loses accuracy on the most structurally distinctive sources, most noticeably the Formula 1 telemetry fold. A continuous learning extension should combine lightweight drift detectors, scheduled retraining on a sliding window of recent arrays, and importance-weighted fine-tuning that biases the loss toward the current operating distribution without discarding the broad prior learned offline. Adding compact domain-identity embeddings to the feature vector is a further avenue worth exploring, since it would allow the classifier to specialise per workload while still

sharing structure across domains, and would provide a natural mechanism for cold-starting the selector on previously unseen sources.

The central argument of this thesis was that per-instance algorithm selection is useful only when prediction quality, decision overhead, and adaptability are treated as a single design problem. The methodology, results, and analysis presented in the prior chapters provide a concrete blueprint of that principle for sorting on real-world numeric arrays, together with a reproducible artefact and a clear research roadmap. The directions explained above shows how the framework can be extended along the portfolio, adaptation, and generalisation axes to broaden its applicability while preserving the cost-aware evaluation discipline established in this work.

REFERENCES

- Akobre í mí, S., Kodua Wiredu í mí, J., Aabaah í mí, I., Adam Wumpini í mí, U., & Kodua Wiredu, J. (2025). A Unified Framework for Theoretical and Experimental Evaluation of Classical and Modern Sorting Algorithms in Real-Time Systems. *Journal of Information Systems and Informatics*, 7(4), 3143–3185. <https://doi.org/10.63158/JOURNALISI.V7I4.1287>
- Aly, A. G., Jensen, A. E., & ElAarag, H. (2025a). Improving Merge Sort and Quick Sort Performance by Utilizing Alphadev’s Sorting Networks as Base Cases. *ACMSE 2025 - Proceedings of the 2025 ACM Southeast Conference*, 239–244. <https://doi.org/10.1145/3696673.3723083>;CSUBTYPE:STRING:CONFERENCE
- Aly, A. G., Jensen, A. E., & ElAarag, H. (2025b). Improving Merge Sort and Quick Sort Performance by Utilizing Alphadev’s Sorting Networks as Base Cases. *ACMSE 2025 - Proceedings of the 2025 ACM Southeast Conference*, 239–244. <https://doi.org/10.1145/3696673.3723083>;CSUBTYPE:STRING:CONFERENCE
- Amini, A. (2026). TwinArray Sort: An Ultrarapid Conditional Non-Comparison Integer Sorting Algorithm. *Electronics* 2026, Vol. 15, Page 609, 15(3), 609. <https://doi.org/10.3390/ELECTRONICS15030609>
- Antoniadis, A., Coester, C., Elias, M., Polak, A., & Simon, B. (2023). *Mixing Predictions for Online Metric Algorithms* (pp. 969–983). PMLR. <https://proceedings.mlr.press/v202/antoniadis23b.html>
- Bai, X., & Coester, C. (2023). Sorting with Predictions. *Advances in Neural Information Processing Systems*, 36, 26563–26584.
- Díaz de León-Hicks, E., Conant-Pablos, S. E., Ortiz-Bayliss, J. C., & Terashima-Marín, H. (2023). Addressing the Algorithm Selection Problem through an Attention-Based Meta-Learner Approach. *Applied Sciences* 2023, Vol. 13, Page 4601, 13(7), 4601. <https://doi.org/10.3390/APP13074601>
- Gelling, W. C., Nebel, M. E., Smith, B., & Wild, S. (2023). Multiway Powersort. *Proceedings of the Workshop on Algorithm Engineering and Experiments, 2023-January*, 190–200. <https://doi.org/10.1137/1.9781611977561.ch16>
- Ghasemi, E., Jugé, V., Khalighinejad, G., & Yazdanyar, H. (2024). Galloping in Fast-Growth Natural Merge Sorts. *Algorithmica* 2024 87:2, 87(2), 242–291. <https://doi.org/10.1007/S00453-024-01285-6>
- Gill, S., Singh, V., ... P. S.-I. J. of, & 2019, undefined. (n.d.). A comparative study of various sorting algorithms. *Papers.Ssrn.ComSK Gill, VP Singh, P Sharma, D*

- Kumar International Journal of Advanced Studies of Scientific Research*, 2019•papers.Ssrn.Com. Retrieved May 8, 2026, from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3329410
- Goldblum, M., Finzi, M., Rowan, K., & Wilson, A. G. (2023). The No Free Lunch Theorem, Kolmogorov Complexity, and the Role of Inductive Biases in Machine Learning. *Proceedings of Machine Learning Research*, 235, 15788–15808. <https://arxiv.org/pdf/2304.05366>
- Hutter, F., Xu, L., Hoos, H. H., & Leyton-Brown, K. (2014). Algorithm runtime prediction: Methods & evaluation. *Artificial Intelligence*, 206(1), 79–111. <https://doi.org/10.1016/j.artint.2013.10.003>
- Jugé, V. (2024). Adaptive Shivers Sort: An Alternative Sorting Algorithm. *ACM Transactions on Algorithms*, 20(4). <https://doi.org/10.1145/3664195;PAGE:STRING:ARTICLE/CHAPTER>
- Kassabgi, M. K., Ihlayyel, M., & Abu-Naser, S. S. (2025). AI-Enhanced Sorting Techniques: Revolutionizing Data Processing and Analysis. In *International Journal of Academic Engineering Research*. <https://philpapers.org/rec/KASAST>
- Kerschke, P., Hoos, H. H., Neumann, F., & Trautmann, H. (2018). Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27(1), 3–45. https://doi.org/10.1162/EVCO_A_00242
- Mankowitz, D. J., Michi, A., Zhernov, A., Gelmi, M., Selvi, M., Paduraru, C., Leurent, E., Iqbal, S., Lespiau, J. B., Ahern, A., Köppe, T., Millikin, K., Gaffney, S., Elster, S., Broshear, J., Gamble, C., Milan, K., Tung, R., Hwang, M., ... Silver, D. (2023a). Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964), 257–263. <https://doi.org/10.1038/S41586-023-06004-9;TECHMETA>
- Mankowitz, D. J., Michi, A., Zhernov, A., Gelmi, M., Selvi, M., Paduraru, C., Leurent, E., Iqbal, S., Lespiau, J. B., Ahern, A., Köppe, T., Millikin, K., Gaffney, S., Elster, S., Broshear, J., Gamble, C., Milan, K., Tung, R., Hwang, M., ... Silver, D. (2023b). Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964), 257–263. <https://doi.org/10.1038/S41586-023-06004-9;TECHMETA>
- Mohammadagha, M., & Student, P. D. (2025). *Hybridization and Optimization Modeling, Analysis, and Comparative Study of Sorting Algorithms: Adaptive Techniques, Parallelization, for Mergesort, Heapsort, Quicksort, Insertion Sort, Selection Sort, and Bubble Sort*. <https://doi.org/10.31224/4537>

- Pezo, C., Hochbaum, D., Godoy, J., & Asín-Achá, R. (2025). Automatic algorithm selection for Pseudo-Boolean optimization with given computational time limits. *Computers and Operations Research*, 173. <https://doi.org/10.1016/j.cor.2024.106836>
- Raman, A., Karatsenidis, K., Sarkar, S., Olma, M., & Athanassoulis, M. (2023). BoDS: A Benchmark on Data Sortedness. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 13860 LNCS, 17–32. https://doi.org/10.1007/978-3-031-29576-8_2/SAVE-RESEARCH
- Renau, Q., & Hart, E. (2024a). Improving Algorithm-Selectors and Performance-Predictors via Learning Discriminating Training Samples. *GECCO 2024 - Proceedings of the 2024 Genetic and Evolutionary Computation Conference*, 1, 1026–1035. <https://doi.org/10.1145/3638529.3654025;TOPIC:TOPIC:CONFERENCE-COLLECTIONS>
- Renau, Q., & Hart, E. (2024b). Improving Algorithm-Selectors and Performance-Predictors via Learning Discriminating Training Samples. *GECCO 2024 - Proceedings of the 2024 Genetic and Evolutionary Computation Conference*, 1, 1026–1035. <https://doi.org/10.1145/3638529.3654025;TOPIC:TOPIC:CONFERENCE-COLLECTIONS>
- Rob, M. A., Hossen, Z., Hossen, K., Ali, M., & Roy, B. (2026a). Wall-L merge sort: A tunable and adaptive sorting algorithm for diverse computing environments. *PLOS ONE*, 21(2), e0341993. <https://doi.org/10.1371/JOURNAL.PONE.0341993>
- Rob, M. A., Hossen, Z., Hossen, K., Ali, M., & Roy, B. (2026b). Wall-L merge sort: A tunable and adaptive sorting algorithm for diverse computing environments. *PLOS ONE*, 21(2), e0341993. <https://doi.org/10.1371/JOURNAL.PONE.0341993>
- Salihu, A., Hoti, M., & Hoti, A. (2022). A Review of Performance and Complexity on Sorting Algorithms. *Proceedings - 2022 International Conference on Computing, Networking, Telecommunications and Engineering Sciences Applications, ConTESA* 2022, 45–50. <https://doi.org/10.1109/CONTESA57046.2022.10011382>
- Shavit, H., & Hoos, H. H. (2024). Revisiting SATZilla Features in 2024. *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*, 305, 27:1-27:26. <https://doi.org/10.4230/LIPICS.SAT.2024.27>

- Smith-Miles, K., & Muñoz, M. A. (2023). Instance Space Analysis for Algorithm Testing: Methodology and Software Tools. *ACM Computing Surveys*, 55(12). <https://doi.org/10.1145/3572895;WGROUP:STRING:ACM>
- Sundaramoorthy, S., & Karunanidhi, G. (2025a). A systematic analysis on performance and computational complexity of sorting algorithms. *Discover Computing* 2025 28:1, 28(1), 250-. <https://doi.org/10.1007/S10791-025-09724-W>
- Sundaramoorthy, S., & Karunanidhi, G. (2025b). A systematic analysis on performance and computational complexity of sorting algorithms. *Discover Computing* 2025 28:1, 28(1), 250-. <https://doi.org/10.1007/S10791-025-09724-W>
- Tanabe, R. (2022). Benchmarking Feature-Based Algorithm Selection Systems for Black-Box Numerical Optimization. *IEEE Transactions on Evolutionary Computation*, 26(6), 1321–1335. <https://doi.org/10.1109/TEVC.2022.3169770>
- Tornede, A., Wever, M., & Hüllermeier, E. (n.d.). *Towards Meta-Algorithm Selection*.
- Uddin, S., & Lu, H. (2024). Dataset meta-level and statistical features affect machine learning performance. *Scientific Reports* 2024 14:1, 14(1), 1670-. <https://doi.org/10.1038/s41598-024-51825-x>
- Wiredu, J. K., Akobre, S., Aabaah, I., & Wumpini, U. A. (2025). A Unified Framework for Theoretical and Experimental Evaluation of Classical and Modern Sorting Algorithms in Real-Time Systems. *Journal of Information Systems and Informatics*, 7(4), 3143–3185. <https://doi.org/10.63158/JOURNALISI.V7I4.1287>
- Wiredu, J. K., Baagyere, E. Y., Nakpih, C., & Aabaah, I. (2025). A Novel Proximity-based Sorting Algorithm for Real-Time Numerical Data Streams and Big Data Applications. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.5249957>
- Xu, S. ;, Liu, W. ;, Wu, C. ;, Li, J., Xu, S., Liu, W., Wu, C., & Li, J. (2024). CNN-HT: A Two-Stage Algorithm Selection Framework. *Entropy* 2024, Vol. 26, Page 262, 26(3), 262. <https://doi.org/10.3390/E26030262>
- Zhang, Z., Chételat, D., Cotnareanu, J., Ghose, A., Xiao, W., Zhen, H. L., Zhang, Y., Hao, J., Coates, M., & Yuan, M. (2024). GraSS: Combining Graph Neural Networks with Expert Knowledge for SAT Solver Selection. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1, 6301–6311. <https://doi.org/10.1145/3637528.3671627;WGROUP:STRING:ACM>